A Major Project Report on

QR CODE BASED SECURE ONLINE VOTING SYSTEM

submitted in partial fulfillment for award of the degree of Bachelor of Technology in Information Technology

By

Sama Ruchitha Reddy	**21321A1276**
Jella Shivani	**21321A1299**
Gadala Surabhi	**21321A12C3**

Under the Esteemed Guidance of

Internal Guide

G Jyothi

Associate Professor, Information Technology

Bhoj Reddy Engineering College for Women

Department of Information Technology

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade, Approved by AICTE and Affiliated to JNTUH) Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad – 500 059, Telangana. www.brecw.ac.in

Ph: +91 -40-2453 7282; Email: principal@brecw.ac.in

AY 2024–25

Bhoj Reddy Engineering College for Women
Department of Information Technology
(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade,
Approved by AICTE and Affiliated to JNTUH)
Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad – 500 059, Telangana. www.brecw.ac.in

Ref No: BRECW/IT Dept/Project 2024-25/PJ-B-11 Date: 11/06/2025

CERTIFICATE

This is to certify that the Major Project entitled **" QR CODE BASED SECURE ONLINE VOTING SYSTEM "** is a bonafide work carried out by

Sama Ruchitha Reddy	**21321A1276**
Jella Shivani	**21321A1299**
Gadala Surabhi	**21321A12C3**

In partial fulfillment for award of the degree of Bachelor of Technology in Department of Information Technology from Bhoj Reddy Engineering College for Women, Hyderabad affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH).

G Jyothi InternalGuide	Dr C Murugamani Head of the Department	External Examiner

ACKNOWLEDGEMENT

It is our pleasure to express our whole hearted thanks to our internal guide **G Jyothi , Associate Professor, Department of Information Technology**, for her extreme guidance and support in completing this project successfully.

We are thankful to our project coordinator **D Navaneetha, Associate Professor, Department of Information Technology,** for her dynamic valuable guidance and constant management.

We express thanks and gratitude to **Dr C Murugamani, Professor & Head of the Department, Information Technology,** for his encouragement and guidance in carrying out the major project presentation.

We would also like to thank **Dr J Madhavan, Professor & Principal of Bhoj Reddy Engineering College for Women** for encouragement in carrying out our major project successfully. We are also thankful to the staff members of Information Technology department, my friends and to our parents who helped us in completing this project successfully.

By

Sama Ruchitha Reddy (21321A1276)

Jella Shivani (21321A1299)

Gadala Surabhi (21321A12C3)

Index

Contents	Page No
List of Tables	ii
Abstract	iii
1. Introduction	**1-7**
1.1 Purpose of the Project	4
1.2 Existing System	4
1.3 Problems in Existing System	5
1.4 Proposed System	5
1.5 Advantages of Proposed System	6-7
2. Related Work	**8-10**
2.1 Survey	9
3. Requirement Analysis	**11-16**
3.1 Functional Requirements	12
3.2 Non-Functional Requirements	13
3.3 Tools/Technologies Required	14
3.4 Computational Requirements	16
4. Design	**17-33**
4.1 Architecture	18-21
4.1.1 System Architecture	18
4.1.2 Technical Architecture	20
4.2 Design	
4.2.1 Data Flow Diagram	22
4.2.2 Uml Diagrams	25
4.3 Modules	33
5. Implementation	**34-45**
5.1 Technologies	34-37
5.2 Code	38-45

6. Screenshots **46-53**

7. Testing **54-60**

7.1 Test Cases 54

7.2 Types of Testing 58

7.3 Result Analysis 60

8. Conclusion **61-62**

9. Future Scope **63-64**

10. References **65-66**

List of Figures/Screenshots

S. No	Figure Name	Fig. No	Page No
1	System Architecture	4.1	18
2	Technical Architecture	4.2	21
3	User Dataflow Diagram	4.2.1.1	23
4	Admin Dataflow Diagram	4.2.1.2	24
5	Use case diagram	4.2.3.1	27
6	Class Diagram	4.2.4.1	28
7	Sequence Diagram	4.2.5.1	29
8	Activity Diagram	4.2.6.1	30
9	Screenshots	6.1-6.14	44-65

List of Tables

S. No	Table Name	Page No
1	Test Cases	58

Abstract

In modern democratic processes, ensuring secure, accessible, and efficient voting mechanisms is imperative. The advent of technology has paved the way for innovative solutions to enhance the voting experience while addressing concerns regarding security and accessibility. This project proposes a novel "QR Code based Secure Online Voting System" developed using Java for the backend, JSP (Java Server Pages), HTML, CSS, and JavaScript for the frontend and MySQL for the database management. The system leverages QR (Quick Response) codes, a two-dimensional barcode technology, to facilitate secure and seamless online voting. QR codes serve as a means to authenticate users, transmit encrypted data, and ensure the integrity of the voting process. By integrating QR code technology, the system enhances security measures and mitigates the risks associated with traditional online voting systems. The frontend of the system, comprising JSP, HTML, CSS, and JavaScript, provides an intuitive and user-friendly interface for voters to access the platform from various devices and browsers. Through responsive design and interactive elements, users can navigate the voting process effortlessly, thereby promoting inclusivity and accessibility. The backend, developed in Java, orchestrates the core functionalities of the system, including user authentication, QR Generation, and database operations. Utilizing Java's robust features and libraries, the backend ensures the reliability, scalability, and performance of the voting system. The MySQL database serves as a secure repository for storing critical data such as user credentials, voting records, and ballot information. Employing MySQL's relational database management capabilities, the system maintains data integrity, confidentiality, and availability throughout the voting lifecycle. By harnessing the power of Java, JSP, HTML, CSS, JavaScript, and MySQL, this project delivers a robust, secure, and user-centric online voting solution. The integration of QR code technology enhances the trustworthiness and integrity of the voting process, thereby contributing to the advancement of democratic practices in the digital age.

Keywords: QR Code Authentication, Secure Online Voting, Java Web Development, User Accessibility.

1. Introduction

1. Introduction

In the digital era, the concept of voting has evolved significantly, leveraging advancements in technology to enhance the efficiency, accessibility, and security of electoral processes. An online voting system represents a modern approach to conducting elections, allowing voters to cast their ballots electronically from remote locations using internet-connected devices. This innovation addresses several limitations of traditional paper-based voting systems, including logistical challenges, time constraints, and accessibility barriers.

Online voting systems, also known as e-voting systems, utilize secure online platforms to facilitate the entire voting process from voter registration and authentication to ballot casting and result tabulation. These systems are designed to ensure that the voting process is transparent, and secure, maintaining the integrity of the election and the anonymity of voters. By incorporating various technologies such as QR codes, encryption, and secure databases, online voting systems aim to provide a seamless and trustworthy voting experience.

One of the primary benefits of an online voting system is its ability to increase voter participation. By enabling voters to cast their ballots from the convenience of their homes or other locations, these systems remove geographical and physical barriers that can prevent individuals from voting This is particularly beneficial for individuals with disabilities, those living in remote areas, and expatriates who might otherwise face significant challenges in participating in elections.

Moreover, online voting systems offer improved efficiency and accuracy in vote counting and result reporting. Automated processes reduce the likelihood of human error in ballot counting and expedite the delivery of election results. This speed and accuracy enhance public confidence in the electoral process and ensure that outcomes are determined swiftly and correctly.

Security is a critical concern in the design of online voting systems. Robust security measures, including user authentication, data encryption, and secure transmission protocols, are implemented to protect against fraud, hacking, and other malicious activities. These systems also incorporate audit trails and verification mechanisms to ensure that each vote is correctly recorded and counted.

The transition to online voting systems also presents opportunities for innovation in electoral processes. Features such as real-time voter education, instant feedback, and interactive interfaces can enhance voter engagement and knowledge. Additionally, the ability to integrate with government databases and other official records can streamline voter verification and registration processes, making elections more efficient and reliable. The system also offers real-time monitoring capabilities for election officials.

They can track voter turnout, system performance, and any anomalies in real-time, enabling prompt responses to potential issues. This level of transparency strengthens public trust in the voting process. One of the significant advantages of this system is its potential to increase voter participation. By enabling remote voting through secure authentication, it removes common barriers such as physical distance, mobility issues, and time constraints. This is particularly beneficial for citizens living abroad or those with limited access to polling stations.

The QR Code-Based Scanner Online Voting System is also cost-effective. It reduces the need for physical ballots, large numbers of polling staff, and the transportation of election materials. This makes it an attractive option for both large-scale national elections and smaller organizational votes. Scalability is another key benefit. The system can handle elections of varying sizes, from local government councils to nationwide parliamentary contests.

Despite its numerous advantages, implementing such a system does come with challenges. Ensuring digital literacy among voters, maintaining system robustness against cyber threats, and adhering to legal frameworks are crucial factors that must be addressed. Comprehensive testing and pilot programs can help mitigate these challenges. The success of a QR Code-Based Scanner Online Voting System relies heavily on public awareness and education.

In conclusion, online voting systems represent a significant advancement in the way elections are conducted, offering numerous benefits over traditional methods. By leveraging technology to enhance accessibility, efficiency, and security, these systems have the potential to transform the democratic process, making it more inclusive and effective for voters worldwide.

1.1 Purpose of project:

The purpose of the QR Code Based Secure Online Voting System project is to develop a secure, scalable, and inclusive platform that enhances the security, accessibility, and efficiency of the voting process. By integrating QR code technology with a robust backend in Java and a user-friendly frontend using JSP, HTML, CSS, and JavaScript, the system aims to modernize digital voting while addressing key concerns such as user authentication, data integrity, and accessibility. This project promotes trust and participation in democratic processes through a technology-driven approach that aligns with contemporary security and usability standards.

A key objective of this project is to strengthen the credibility and transparency of online voting by employing QR code-based user verification and secure data handling mechanisms. Traditional digital voting systems often struggle with issues like identity fraud and unauthorized access. This system mitigates such risks through reliable authentication protocols and encrypted data transactions, ensuring that each vote is both legitimate and confidential. The backend architecture supports scalability to accommodate large-scale elections, making the system suitable for institutions, organizations, and governmental use.

Additionally, the project emphasizes accessibility and ease of use by offering a web-based interface developed with familiar web technologies. This ensures that users with varying levels of digital literacy can confidently participate in the voting process. The integration of QR codes simplifies the login process, reduces the need for manual credentials, and minimizes user errors. By combining advanced technology with a focus on user experience, the QR Code Based Secure Online Voting System contributes to a broader goal of promoting fair, secure, and inclusive electoral participation in the digital age.

1.2 Existing System:

The existing election system primarily relies on traditional in-person voting methods, where voters must visit designated polling stations to cast their ballots using physical paper. Voter identity is verified through government-issued IDs, and election officials oversee the process to ensure integrity and privacy. After voting, ballots are manually collected and transported for counting, which involves sorting and tallying votes under the supervision of election officials to maintain transparency.

1.3 Problems in Existing System:

- Limited Accessibility: Traditional voting requires in-person attendance, which excludes elderly, disabled, or remote voters.
- Time-Consuming Process**:** Manual voter verification, voting, and counting slow down the entire election workflow.
- Prone to Human Errors: Physical handling of ballots can lead to miscounts, invalid entries, and misplaced votes.
- Security Vulnerabilities: Paper ballots are susceptible to tampering, damage, or loss during storage and transport.
- High Operational Costs**:** Running physical elections demands logistics, paper, staff, and secure transport — all costly.
- Low Voter Engagement: Long lines, travel burdens, and rigid schedules discourage people from voting.
- Environmental Concerns**:** Paper-based elections generate substantial waste and are not eco-friendly.

1.4 Proposed System:

The proposed system, "QR Code based Secure Online Voting System," aims to revolutionize the voting process by leveraging modern technology to overcome the limitations of traditional in-person voting methods. This system offers a seamless, secure, and accessible platform for voters to cast their votes remotely, eliminating the need for physical polling stations and paper-based ballots. The proposed system utilizes QR (Quick Response) codes as a secure means of authenticating voters.

Each voter is assigned a unique QR code that serves as their digital identity during the voting process. QR codes provide a convenient and efficient method for voter verification, reducing the risk of fraud and ensuring that only authorized individuals can participate in the election. The proposed system features an intuitive and user-friendly online voting platform accessible via web browsers. Voters can access the platform from anywhere with an internet connection, eliminating the need to travel to physical polling stations and accommodating individuals with mobility issues or remote locations.

The proposed system is developed using Java, the backend of the system orchestrates essential functions such as user authentication, ballot generation, and database interactions. Java's robust features ensure the reliability, scalability, and performance of the voting system, enabling smooth and efficient voting operations even during high traffic periods. The frontend of the system is built using JSP (JavaServer Pages), HTML, CSS, and JavaScript to provide an interactive and user-friendly interface for voters. Through responsive design and intuitive navigation, voters can easily navigate the voting process, select their preferred candidates, and submit their votes securely.

The proposed QR Code Based Secure Online Voting System incorporates robust security measures, including encryption techniques, to protect voter data and ensure secure transmission between devices and the voting platform. Designed for scalability and flexibility, it can accommodate elections of various sizes, from local polls to national campaigns. The system maintains a comprehensive audit trail, allowing election administrators to track participation, monitor submissions, and detect anomalies in real-time, ensuring transparency and accountability. By leveraging advanced technologies, it modernizes the voting process, providing a secure, accessible, and efficient platform for democratic participation. This approach enhances electoral integrity, promotes inclusivity, and offers a reliable alternative to traditional in-person voting methods.

1.5 Advantages of Proposed Systems:

- Expanded Accessibility**:** Allows voters to cast ballots remotely from any location with an internet connection, increasing participation among individuals with mobility challenges, remote residents, and those with work or personal constraints.
- Improved Convenience: Enables voters to participate in elections at any time, eliminating restrictions imposed by specific voting hours or locations. The 24/7 availability of the platform enhances voter satisfaction and engagement.
- Optimized Cost-Efficiency: Reduces election costs by eliminating physical polling stations, paper-based ballots, and associated logistical expenses. Lower staffing, transportation, and administrative overhead improve overall cost-effectiveness.
- Stronger Security: Ensures voting integrity through QR code authentication and encryption techniques. Unique QR codes act as secure digital identities, reducing fraud risks while encryption safeguards sensitive voter information.3

- Real-Time Election Results: Provides instant vote tabulation, enabling faster and more accurate reporting of results. This transparency fosters public trust and minimizes uncertainty surrounding election outcomes.
- Scalability and Flexibility: Supports elections of varying sizes, from local polls to national campaigns, ensuring adaptability to different jurisdictions and voter populations. The system efficiently handles high user loads without compromising performance.
- Environmental Sustainability: Eliminates paper ballots, reducing ecological footprint and promoting resource conservation. The shift to electronic voting aligns with global sustainability initiatives.
- User-Friendly Design: Offers an intuitive interface with responsive design, clear instructions, and interactive features to facilitate easy navigation, ensuring accessibility for all voter demographics.
- Comprehensive Audit and Transparency: Maintains detailed audit trails of all voting activities, enabling election administrators to track participation, monitor submissions, and detect irregularities in real-time for enhanced electoral integrity.

2. Related Work

2.Related Work

2.1 Survey:

Lokhande et al., in their paper *"E-Voting through Biometrics and Cryptography Steganography Technique with Conjunction of GSM Modem"*, explore a security-enhanced e-voting system that integrates biometrics, cryptography, and steganography. They propose a mechanism where a secret key is merged with a cover image using biometric measures like fingerprint recognition, resulting in a stego image that appears identical to the original but remains undetectable to the human eye. The authors highlight the necessity of cryptographic techniques in securing voter authentication, emphasizing how combining steganography and encryption can significantly reduce election fraud risks. Their study presents a robust approach to enhancing voter security by ensuring authentication through dual verification layers, making hacking nearly impossible.

Patel Cetinkaya et al., in their paper *"Practical Aspects of DynaVote e-Voting Protocol"*, examine the feasibility and effectiveness of the DynaVote e-Voting protocol, which eliminates the need for complex encryption algorithms while maintaining core voting requirements like privacy, accuracy, and individual verifiability. The study presents a prototype implementation that leverages the Pseudo-Voter Identity (PVID) mechanism, ensuring unlinkability and anonymous voting while maintaining security. The authors highlight how their protocol enables efficient voter authentication and participation without physical assumptions like untappable communication channels. Their findings suggest that DynaVote is a practical and applicable solution for real-world electronic elections, providing transparency, mobility, and reliability in network-based voting systems.

Kumar et al., in their paper *"Design a Secure Electronic Voting System Using Fingerprint Technique"*, address security vulnerabilities in traditional paper-based voting systems and propose an e-voting solution that integrates fingerprint biometrics with advanced encryption techniques. Their research aims to eliminate fraudulent activities such as voter impersonation and ballot stuffing by leveraging biometric authentication combined with cryptographic watermarking. The authors emphasize how the proposed system ensures vote integrity, confidentiality, and secure storage, making it suitable for deployment in digital elections.

Abdolahi et al., in their paper *"Multimodal Biometric System Fusion Using Fingerprint and Iris with Fuzzy Logic"*, investigate the limitations of single biometric authentication and propose a multimodal biometric system that combines fingerprint and iris recognition with fuzzy logic-based fusion techniques. Their research demonstrates that using multiple biometric modalities significantly improves authentication accuracy while mitigating issues such as spoof attacks and non-universality in single biometrics. The authors introduce a weighted decision approach that refines security and verification processes, ensuring robust voter authentication in electronic voting platforms. Their findings highlight the advantages of multimodal biometrics in strengthening security and reducing authentication errors in digital voting systems.

Nikita et al., in their paper *"Secure Online Voting System Proposed By Biometric And Steganography"*, present an innovative approach to securing online voting systems by integrating biometric authentication with cryptography and steganography. Their study showcases how steganography conceals sensitive voter credentials within images, preventing unauthorized access while ensuring secure vote transmission. By combining encryption techniques with secret key mechanisms, they establish a highly secure voter authentication process that minimizes the likelihood of hacking attempts. The authors conclude that their security framework enhances voter trust and safeguards the integrity of the electoral process, making it a viable solution for secure digital elections.

3. Requirement Analysis

3.Requirement Analysis

Taking into account the comparative analysis stated in the previous section we could start specifying the requirements that our website should achieve. As a basis, an article on all the different requirements for software development was taken into account during this process. We divide the requirements in 2 types: functional and non- functional requirements.

3.1.1 Functional Requirements:

- **Voter Authentication:** The system ensures secure and reliable verification of voters using unique QR codes. Before accessing the voting platform, voters must authenticate their identity through QR-based validation, reducing risks of fraudulent voting and unauthorized participation.

- **Secure Access:** A comprehensive security framework is implemented, including encryption mechanisms and controlled access protocols. These security measures safeguard the system from unauthorized access, data breaches, and tampering, ensuring integrity throughout the voting process.

- **Admin Dashboard**: The system The system automatically counts, verifies, and organizes votes with high accuracy, ensuring a transparent and fair election process. The automated vote tabulation mechanism eliminates human errors and accelerates result computation, leading to faster election outcomes.

- **Vote Tabulation**: The system automatically counts, verifies, and organizes votes with high accuracy, ensuring a transparent and fair election process. The automated vote tabulation mechanism eliminates human errors and accelerates result computation, leading to faster election outcomes.

- **Data Encryption:** Strong encryption techniques are employed to protect sensitive voter data, ensuring that all transmitted and stored voting information remains confidential and tamper-proof. These cryptographic safeguards enhance security, maintaining the integrity of all votes cast.

3.2 Non-Functional Requirements:

- **Performance**: The system is designed to operate efficiently even under high demand, ensuring minimal latency in voter authentication and vote tabulation. Optimized algorithms and load balancing techniques enable smooth processing during peak voting periods. By leveraging high-performance server configurations, the system maintains speed and responsiveness, guaranteeing seamless interaction for both voters and administrators.

- **Usability**: The voting platform prioritizes an intuitive and user-friendly interface, allowing users of all technical backgrounds to navigate the system effortlessly. Clear instructions, accessible design elements, and responsive layouts ensure a smooth voting experience. Whether accessed via desktop or mobile, the system supports multi-device compatibility, making online voting convenient and accessible.

- **Reliability**: The system ensures high availability and fault tolerance, maintaining uninterrupted service throughout the voting period. Redundant server architectures and real-time backup mechanisms safeguard against failures, preventing data loss and system downtime. By implementing continuous monitoring and recovery protocols, the platform guarantees consistent access for voters and administrators.

- **Security**: Robust encryption, multi-layer authentication, and secure data handling mechanisms protect sensitive voter information from unauthorized access. The system employs end-to-end encryption for vote transactions and user credentials, ensuring data integrity and confidentiality. Additionally, secure access controls and cybersecurity best practices minimize vulnerabilities, fortifying the system against potential threats.

- **Scalability:** The system is designed to support large-scale elections, accommodating multiple concurrent users and increased data volumes without performance degradation. Whether handling local, national, or organizational voting scenarios, the infrastructure adapts dynamically to fluctuating user activity. Advanced cloud-based solutions and elastic resource management ensure optimal system performance, making it suitable for elections of varying scales.

3.3 Tools/Technologies Requirements:

To effectively implement the QR Code Based Online Voting System, a suite of reliable and proven technologies is used to ensure security, scalability, and usability. The system is developed using Java, a platform-independent and object-oriented programming language, and utilizes MySQL as a robust relational database. Technologies such as JDBC are employed for seamless database interaction, and JFreeChart is integrated for visualization and reporting purposes. Together, these tools support real-time vote authentication, recording, monitoring, and result generation with high accuracy and security.

3.3.1 Programming Languages:

Java: The primary programming language used for backend development. Java's portability and robust libraries make it ideal for building secure and scalable enterprise-level applications like online voting systems. Java's "write once, run anywhere" philosophy ensures system compatibility across platforms.

HTML/CSS/JavaScript: Used to design and implement the frontend interface, enabling user interaction through a browser. These technologies contribute to a responsive and user-friendly voting environment.

3.3.2 Frameworks and Libraries:

Java Servlets and JSP: Used for building the server-side logic of the system, handling voter requests, authentication, and communication between frontend and backend.

JDBC (Java Database Connectivity): Facilitates database connectivity, allowing the application to interact with the MySQL database for reading and writing voter data and vote records.

JFree Chart: A Java charting library used to generate visual representations of voting results, analytics, and administrative reports.

JavaBeans: Utilized for component-based software development and object serialization within the system's modules.

3.3.3 Databases and Storage:

MySQL: A fast and reliable open-source relational database used to store all the structured data related to voters, elections, and voting results. It supports large data volumes, indexing, and efficient querying.

ODBC : Can be integrated for flexible database connectivity across different platforms or legacy systems if needed.

3.3.4 Authentication & Security:

QR Code-Based Voter Authentication: Every voter is assigned a unique QR code which is scanned to verify their identity before granting access to the voting interface.

Secure Session Handling: Java provides session tracking and management to prevent unauthorized voting and ensure that each vote is submitted only once.

JVM Security Features: Java Virtual Machine (JVM) offers bytecode verification and runtime access control, ensuring a secure execution environment.

Role-Based Access Control (RBAC): Differentiates user roles such as voters, administrators, and auditors to manage permissions and access securely.

3.3.5 Development and Deployment Tools:

Apache Tomcat: Serves as the web server and servlet container for running Java web applications.

Git & GitHub: Used for version control, tracking changes in the codebase, and supporting collaborative development.

Maven: A build automation tool that handles project dependencies, compiling, and packaging of the application.

3.3.6 IDEs and Editors:

Eclipse / IntelliJ IDEA: Integrated Development Environments used to write, debug, and manage Java code effectively.

MySQL Workbench: Provides a graphical interface for managing and querying the MySQL database, designing schemas, and administering database tasks.

3.4 Computational Requirements:

Software Requirements:

Operating system	:	Windows 10/11.
Coding Language	:	JAVA.
Frontend	:	JSP, HTML, CSS, JavaScript.
JDK Version	:	JDK 21.
IDE Tool	:	Apache Netbeans IDE 20.
Tomcat Server Version	:	Apache Tomcat 9.0.84
Database	:	MYSQL.

Hardware Requirements:

System	:	Pentium i3 Processor
Hard Disk	:	500 GB.
Monitor	:	15’’ LED
Input Devices	:	Keyboard, Mouse
Ram	:	4 GB

4.Design

4.Design

4.1 Architecture:

4.1.1 System Design:

The QR Code Based Online Voting System is designed to ensure secure, authenticated, and transparent remote voting. The architecture emphasizes voter verification, encrypted vote casting, and feedback integration, ensuring end-to-end integrity and trust. The following modules represent the system components as illustrated in the diagram:

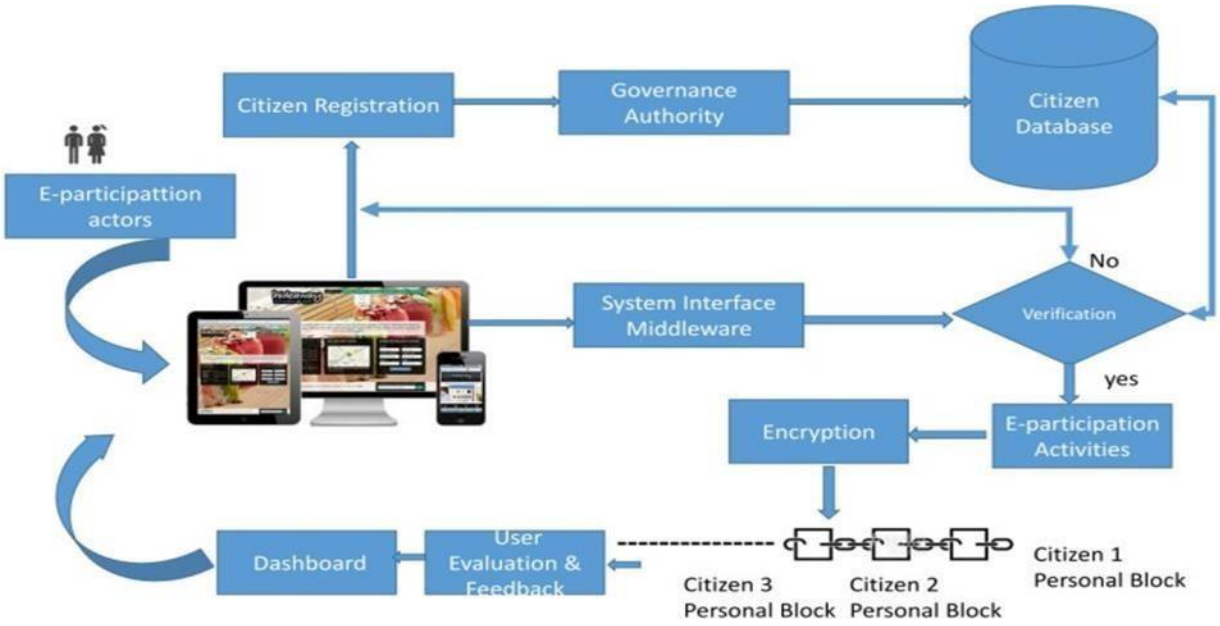


Fig 4.1: System Architecture

4.1.1.1.1 E-Participation Actors:

These are the users (citizens/voters) who interact with the system using various devices such as smartphones, tablets, or desktops. They initiate the voting process by accessing the platform and providing their QR code for verification.

4.1.1.1.2 Citizen Registration:

Citizens must register with the system before participating in the election process. During registration, the system collects voter information and generates a unique QR code for each

individual. This step is managed by the Governance Authority to ensure only eligible citizens are included.

4.1.1.1.3 Governance Authority:

The central authority responsible for approving citizen registrations, managing election data, and validating voter eligibility. It connects the Citizen Registration process with the Citizen Database, ensuring that each registrant is authenticated.

4.1.1.1.4 Citizen Database

The A centralized storage system containing personal, demographic, and authentication data of registered voters. When a QR code is submitted, the system checks this database to verify its validity.

4.1.1.1.5 System Interface Middleware:

This component acts as a communication bridge between the user interface and backend services. It processes the verification request, interacts with the database, and routes users either to the voting process (if verified) or back to the registration process (if not).

4.1.1.1.6 Verification Process:

The system checks the validity of the voter's QR code and registration details. A Yes response allows the user to proceed to voting activities. A No response redirects them to appropriate support or re-registration.

4.1.1.1.7 E-participation Activities:

Upon successful verification, the voter is granted access to the online ballot. Here, they can cast their vote securely. This module also logs the voting event for transparency and audit.

4.1.1.1.8 Encryption:

Before being recorded, all votes are encrypted to ensure confidentiality and tamper resistance. This encryption layer is crucial to preserving the anonymity and integrity of each vote.

4.1.1.1.9 Blockchain Ledger (Citizen Personal Block):

Each vote is recorded on a blockchain structure where every voter gets a unique Personal Block. This ensures immutability and traceability of voting data, preventing double voting or vote alteration.

4.1.1.1.10 Dashboard:

The administrative dashboard offers real-time insights into system performance, user participation, and data analytics. It serves governance bodies for monitoring, auditing, and decision-making.

4.1.1.1.11 Dataflow:

The architecture follows a modular pipeline:

4.1.1.1.11.1 The request is initiated via QR code scan by the voter.

4.1.1.1.11.2 It passes through the middleware and is routed to the verification module.

4.1.1.1.11.3 Verifies data is encrypted and sent to the voting interface.

4.1.1.1.11.4 The vote is cast, encrypted, and stored in the block chain.

4.1.1.1.11.5 Logging and session tracking are managed across all modules.

4.1.2 Technical Architecture:

The architecture enables secure, role-based access for Admins and Users interacting with a centralized database. Admins can authorize users, add election and candidate details, and monitor vote counts through a protected backend interface. Users log in, scan their QR code using a webcam, and upon verification, gain access to cast votes and view results. All actions are recorded in real time, ensuring transparency and data integrity. The system relies on QR-based authentication and secure data flow to facilitate efficient, accountable, and tamper-proof digital elections.

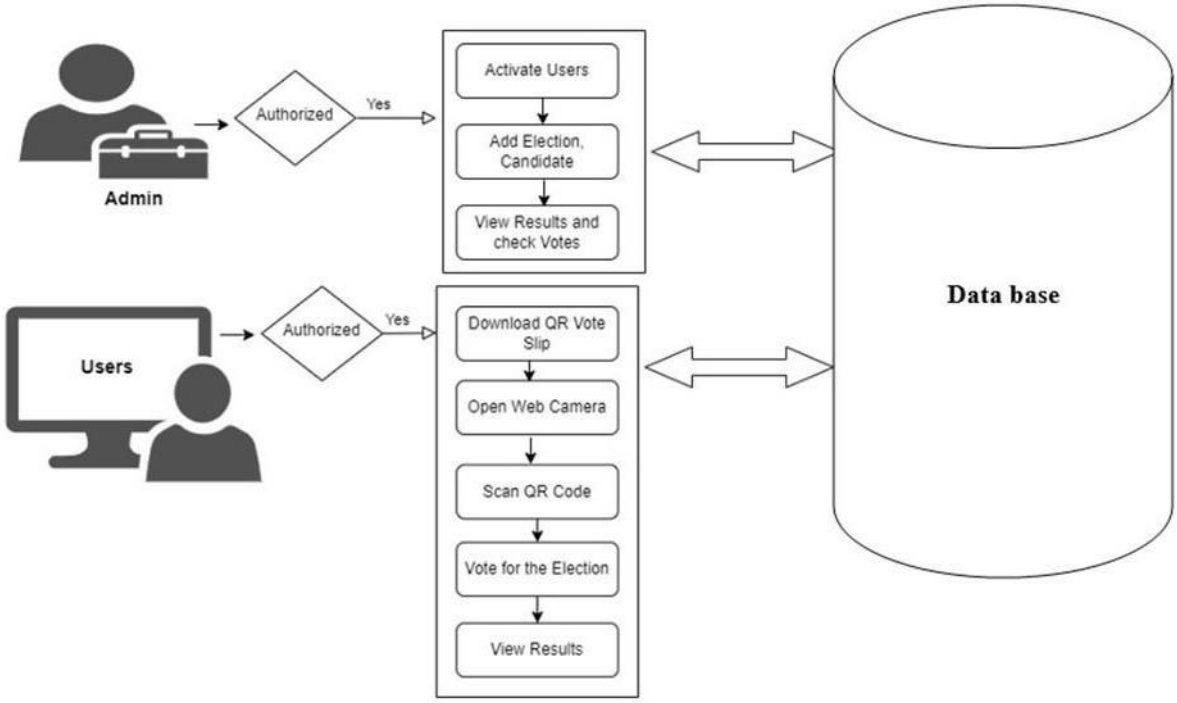


Fig 4.2: Technical Architecture

4.1.2.1 End Users and Interfaces:

End users interact with the system via desktop or web interfaces. Access is granted only after secure authorization.

Admins: operate through a control panel interface to manage election operations.

Users: Access the voting portal to cast votes after scanning their QR code.

4.1.2.2 Admin Services:

This module handles secure administrative functions.it enables admins to

- Activate registered users for voting eligibility.
- Create elections and manage candidate information.
- Monitor vote counts and view real-time results through the connected databases.

4.1.2.3 User Services:

User services guide voters through a secure and validated voting process:

- Download unique QR voting slips.
- Open the webcan interface to scan the QR code.
- Upon successful verification, users are permitted to cast their votes and view election results.

4.1.2.4 Centralized Database System:

The database acts as the core of the system:

- Stores user credentials, election details, candidate data, and vote records.
- Synchronizes real-time data between admin actions and user interactions.
- Ensures integrity, accuracy, and transparency of all electoral processes.

4.2 Design:

4.2.1 Data Flow Diagram:

The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system. The data flow diagram (DFD) is one of the most important modelling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output. DFD is also known as bubble chart. A DFD may be used to represent a systematic any level of abstraction. DFD may be partitioned into levels that representing increasing information flow and functional details.

User:

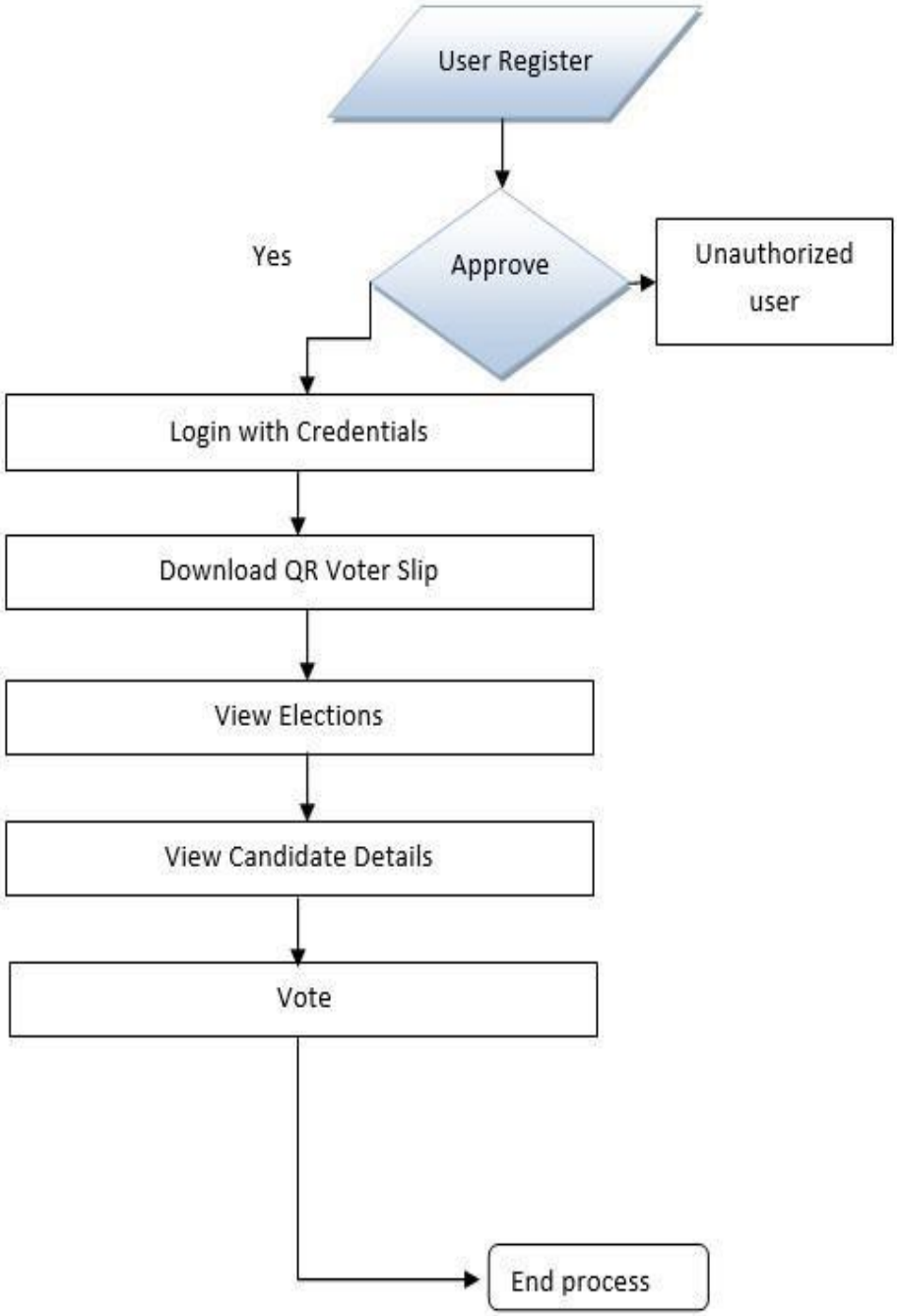


Fig4.2.1.1 Data flow diagram

Admin:

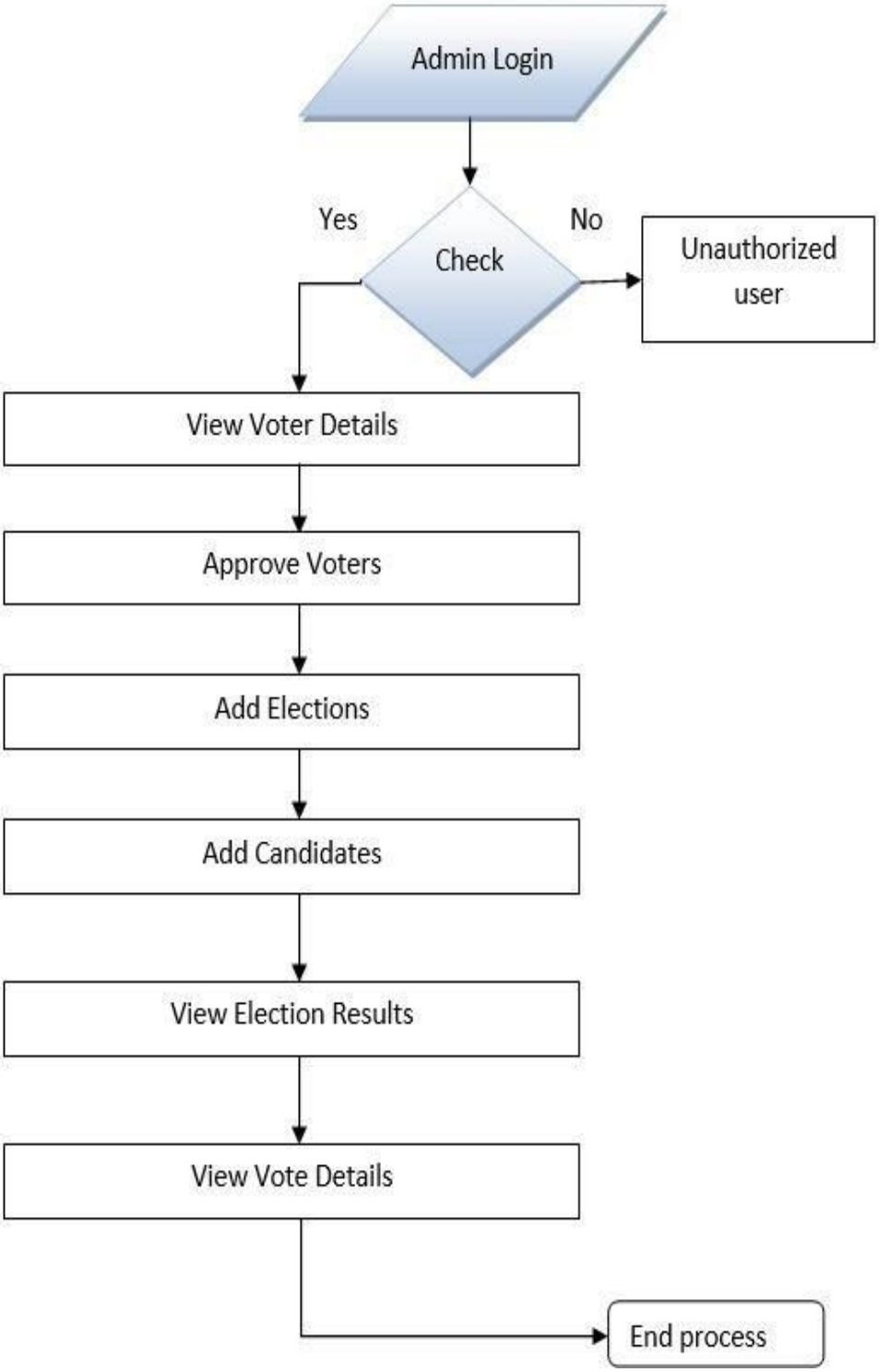


Fig4.2.1.2 Data flow diagram

4.2.2 UML Diagrams:

UML stands for Unified Modeling Language. UML is a standardized general- purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Metamodel and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

There are two broad categories of diagrams and they are again divided into subcategories

- Structural Diagrams
- Behavioural Diagrams

4.2.2.1 Structural Diagrams:

The structural diagrams represent the static aspect of the system. These static aspects represent those parts of a diagram, which forms the main structure and are therefore stable.

These static parts are represented by classes, interfaces, objects, components, and nodes. The four structural diagrams are

- Class Diagarm
- Object Diagram
- Component Diagram
- Deployment Diagram

4.2.2.2 Class Diagram:

Class diagrams are the most common diagrams used in UML. Class diagram consists of classes, interfaces, associations, and collaboration. Class diagrams basically represent the object-oriented view of a system, which is static in nature. Active class issued in a class diagram to represent the concurrency of the system. Class diagram represents the object orientation of a system. Hence, it is generally used for development purpose. This is the most widely used diagram at the time of system construction.

4.2.2.3 Object Diagram:

Object diagrams can be described as an instance of class diagram. Thus, these diagrams are closer to real-life scenarios where we implement a system. Object diagrams are a set of objects and their relationship is just like class diagrams. They also represent the static view of the system. The usage of object diagrams is similar to class diagrams but they are used to build prototype of a system from a practical perspective.

4.2.2.4 Component Diagram:

Component diagrams represent a set of components and their relationships. These components consist of classes, interfaces, or collaborations. Component diagrams represent the implementation view of a system. During the design phase, software artifacts (classes, interfaces, etc.) of a system are arranged in different groups depending upon their relationship. Now, these groups are known as components. Finally, it can be said component diagrams are used to visualize the implementation.

4.2.2.5 Delpoyment Diagram:

Deployment diagrams are a set of nodes and their relationships. These nodes are physical entities where the components are deployed. Deployment diagrams are used for visualizing the deployment view of a system. This is generally used by the deployment team.

4.2.2.6 Behavioural Diagram:

Any system can have two aspects, static and dynamic. So, a model is considered as complete when both the aspects are fully covered. Behavioural diagrams basically capture the dynamic aspect of a system. Dynamic aspect can be further described as the changing/moving parts of a system.

UML has the following five types of behavioral diagrams-

- Use Case Diagram
- Sequence Diagram
- Collaborative Diagram
- Activity Diagram

4.2.3 Use Case Diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

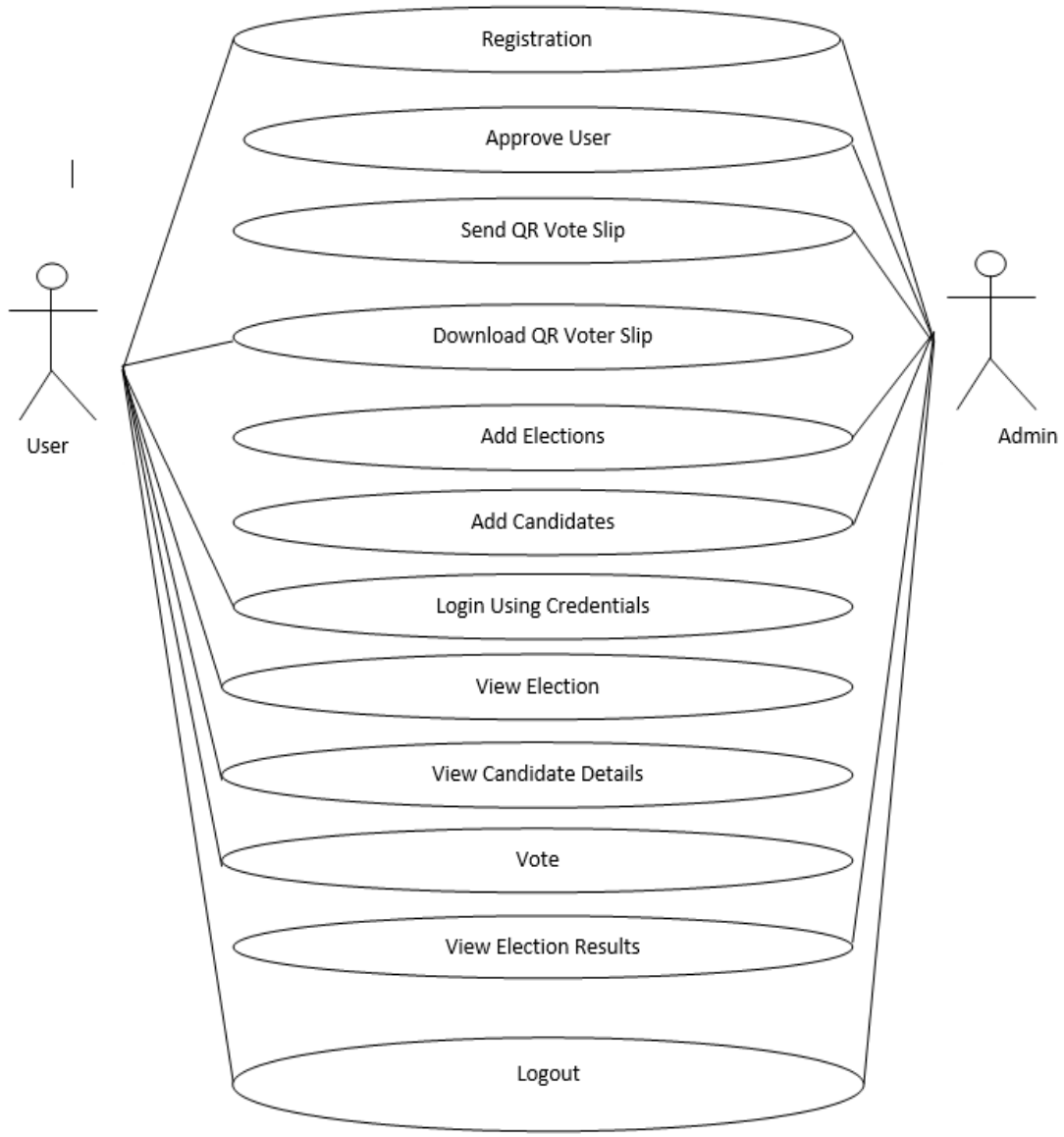


Fig:4.2.3.1 Use case diagram

4.2.4 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

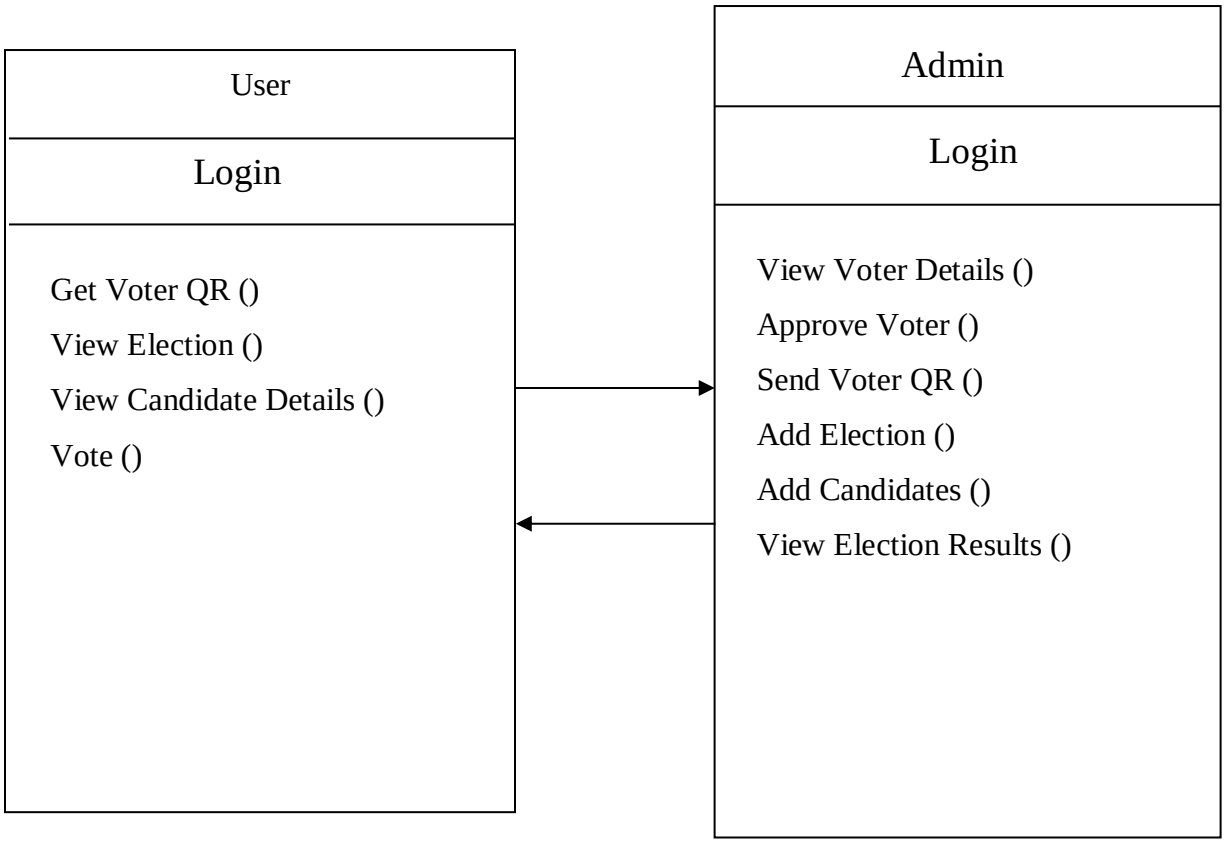


Fig 4.2.4.1 Class Diagram

4.2.5 Sequence Diagram:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

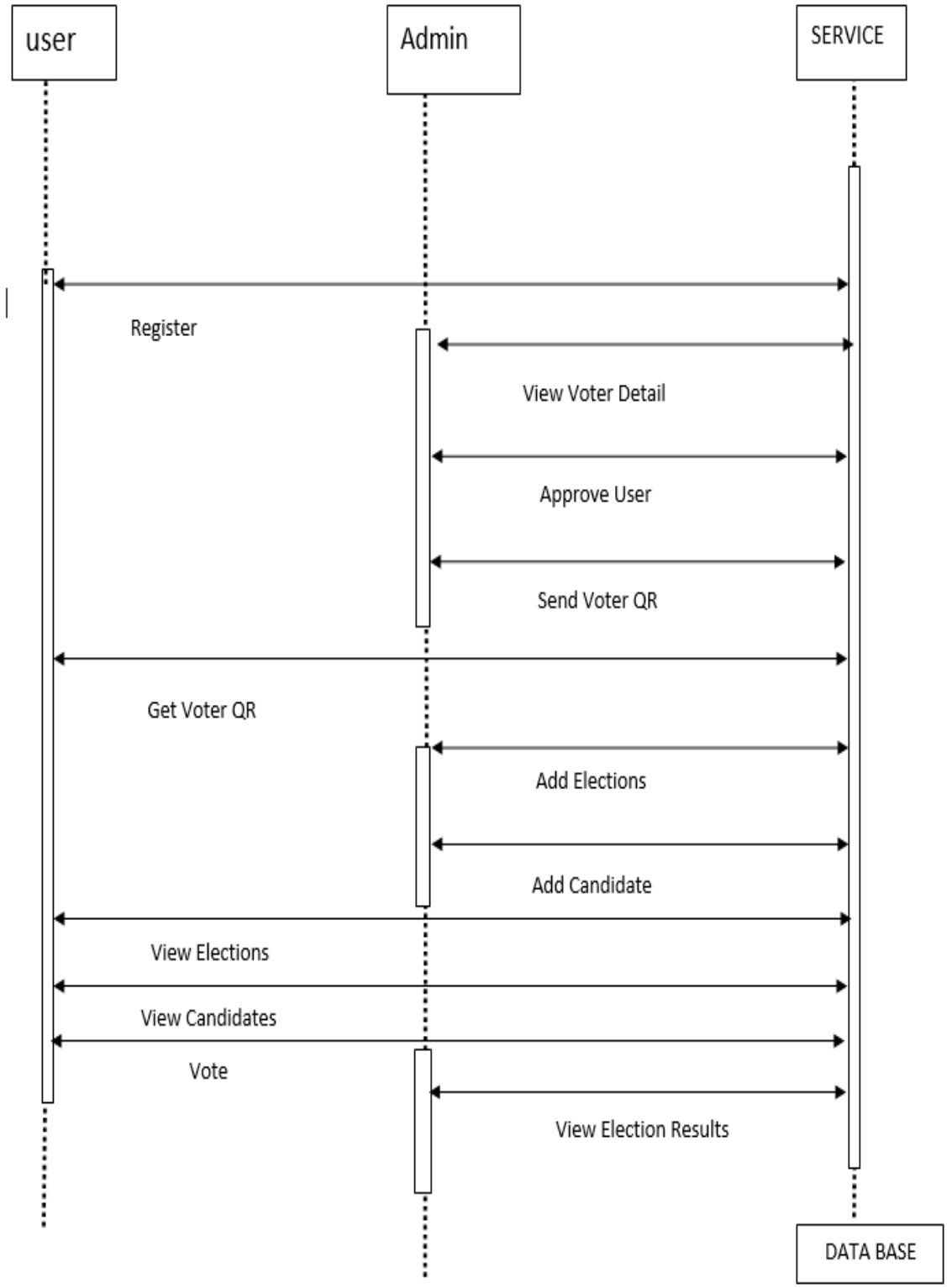


Fig 4.2.5.1 Sequence Diagram

4.2.6 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

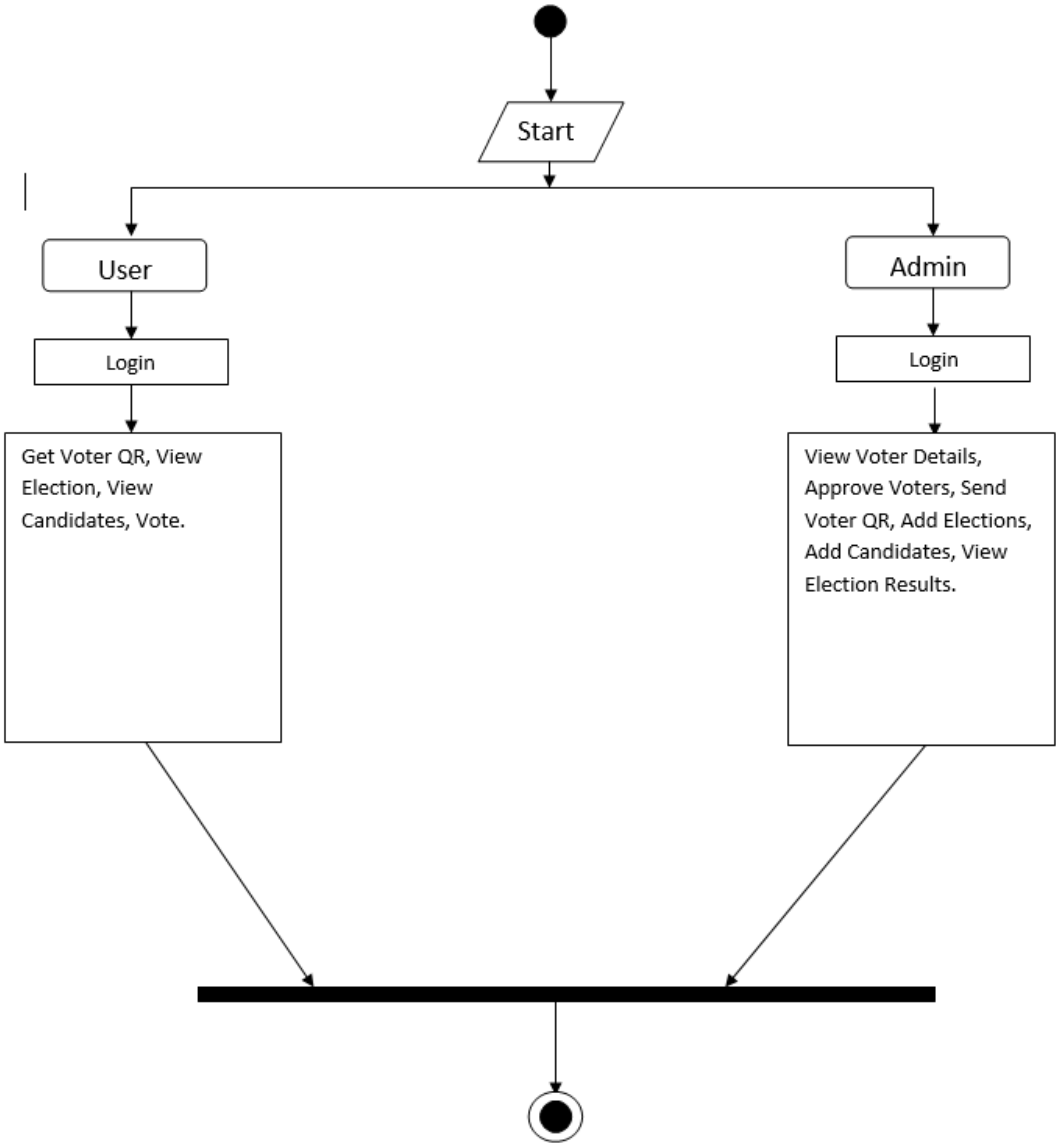


Fig 4.2.6.1 Activity Diagram

4.3 Modules:

Module is a part of a program. Programs are composed of one or more independently developed modules. A module description provides detailed information about a module and its supported components. The included description is available directly by making environment check for supported components. The modules are:

- User Authentication.
- QR Generation and Management.
- Voting Casting and Recording
- Results Tabulation and Reporting

4.3.1 User Authentication:

In the "QR Code based Secure Online Voting System," the User Authentication Module plays a pivotal role in ensuring secure access to the platform while adhering to a stringent registration and approval process. This module encompasses several key functionalities tailored to facilitate user registration, admin approval, and subsequent user authentication. Upon accessing the system, new users are prompted to register by providing their personal details, including name, date of birth, email ID, residential address, and voter ID number. Additionally, users are required to upload a scanned copy of their voter ID card for verification purposes. Following user registration, the system awaits approval from the administrator before granting access to the user. The admin reviews the user's provided information, including the uploaded voter ID card, to verify their identity and eligibility to participate in the voting process. The admin conducts a thorough verification process to ensure the authenticity of the user's details. This may involve cross-referencing the provided information with official voter registration records and verifying the legitimacy of the uploaded voter ID card. Based on the results of the verification process, the admin either approves or rejects the user's registration request. If approved, the user is granted access to the system and notified via email, allowing them to proceed with logging in and participating in the voting process. However, if rejected, the user is informed of the decision and prohibited from accessing the system or casting their vote. Approved users can log in to the system using their registered credentials, including username and password. Upon successful authentication, users gain access to the voting platform, where they can view the available ballots, make their selections, and submit their votes securely. Throughout the registration and approval process, the system maintains a log of all

user activities, including registration attempts, admin approvals or rejections, and user logins. This comprehensive audit trail provides transparency and accountability, allowing administrators to monitor user activities and detect any irregularities or suspicious behaviour.By implementing a rigorous registration and approval process within the User Authentication Module, the "QR Code based Secure Online Voting System" maintains the integrity of the user database and enhances overall security, thereby fostering trust and confidence in the electoral process.

4.3.2 QR Generation and Management:

The QR Generation and Management Module in the "QR Code based Secure Online Voting System" is responsible for generating and managing QR codes that serve as digital identities for registered users. This module encompasses several key functionalities aimed at efficiently generating QR codes, associating them with user profiles, and ensuring their secure management throughout the voting process Once generated, QR codes are associated with user profiles within the system. Each QR code is linked to the corresponding user's account, enabling seamless authentication during the voting process. This association ensures that only authorized users can access the system using their QR codes. The module facilitates the display and distribution of QR codes to registered users. Upon approval, users are provided with their unique QR codes, which can be accessed through the user dashboard. Additionally, users may download and keep it in their phone or may print their QR codes for voting. The QR Generation and Management Module play a crucial role in ensuring the secure and efficient authentication of users within the "QR Code based Secure Online Voting System." By generating unique QR codes, associating them with user profiles, and implementing robust management and validation processes, the module enhances the integrity and reliability of the voting process, thereby fostering trust and confidence among users.

4.3.3 Vote Casting and Recording:

The Vote Casting Module in the "QR Code based Secure Online Voting System" facilitates the secure and transparent casting of votes by registered users. This module incorporates a series of steps to ensure that users can participate in elections only after receiving approval from the administrator and adhere to the principle of one vote per user per election. Once authenticated, users are presented with a list of available elections in which they are eligible to participate. The list includes details such as the election name, date, and description. Users can select the desired election from the list to proceed with the voting process. After selecting the election, users are presented with a list of candidates contesting in the selected election.Users

can select their preferred candidate for whom they wish to cast their vote. Upon selecting a candidate, users confirm their choice and submit their vote electronically. The system implements validation checks to ensure that users can only vote once for each election. If a user attempts to vote again for the same election, the system detects the duplicate vote and prevents it from being cast. Users are notified that they have already voted in the election and cannot vote again. The module maintains a comprehensive audit trail of all voting activities, including user authentication, election selection, candidate selection, and vote submission. Audit logs record relevant metadata such as user ID, election ID, timestamp, and voting outcome, providing administrators with visibility into voting activities and facilitating accountability and transparency. By implementing the Vote Casting Module, the "QR Code based Secure Online Voting System" ensures that registered users can securely cast their votes in elections only after receiving approval from the administrator. By adhering to the principle of one vote per user per election and maintaining a transparent audit trail, the module enhances the integrity and reliability of the online voting process, thereby fostering trust and confidence among users.

4.3.4 Results Tabulation and Reporting:

The Results Tabulation and Reporting Module is a critical component of the "QR Code based Secure Online Voting System," responsible for aggregating, tabulating, and presenting the election results to both users and administrators. This module ensures the accurate counting of votes, provides comprehensive reporting features, and supports transparency and accountability in the electoral process. In addition to the general results available to users, administrators have access to more detailed reporting features. Admin can view a comprehensive breakdown of the votes, including which users voted for which candidates. This capability is crucial for verifying the integrity of the voting process and for conducting post-election audits. The system ensures that this sensitive information is accessible only to authorized administrators, maintaining the confidentiality and security of voter data. By implementing the Results Tabulation and Reporting Module, the "QR Code based Secure Online Voting System" ensures that election results are accurately calculated, transparently reported, and securely stored. This module not only provides immediate feedback to voters and administrators but also upholds the integrity and trustworthiness of the electoral process through robust security and detailed auditing capabilities.

5. Implementation

5.Implementation

5.1Technologies:

The development of the "QR Code Based Secure Online Voting System" integrates multiple technologies and platforms to ensure robust, secure, and scalable functionality across the application. The system has been primarily built using Java, supported by Java Database Connectivity (JDBC), MySQL for database management, ODBC for broader compatibility, and JFreeChart for result visualization. These technologies together form a cohesive development environment that allows for the creation of a reliable, interactive, and real- time web-based application capable of handling sensitive voting data with

- **Java Technology:**

Java is a high-level, object-oriented, and platform-independent programming language developed by Sun Microsystems. It provides a powerful and flexible environment suitable for building robust, scalable, and distributed applications. Java follows the principle of "write once, run anywhere," meaning its compiled bytecode can run on any platform equipped with a Java Virtual Machine (JVM). This platform independence makes Java an ideal choice for cross-platform applications such as the QR Code-Based Secure Online Voting System. Java supports a wide variety of programming features including inheritance, encapsulation, polymorphism, and abstraction. These object-oriented principles help in organizing complex systems into modular, reusable code blocks. Java's memory management, via automatic garbage collection, reduces memory leaks and enhances stability. Its multithreading capabilities allow simultaneous execution of tasks, essential for real-time systems like online. Voting System. Java supports a wide variety of programming features including inheritance, encapsulation, polymorphism, and abstraction. These object-oriented principles help in organizing complex systems into modular, reusable code blocks. Java's memory management, via automatic garbage collection, reduces memory leaks and enhances stability. Its multithreading capabilities allow simultaneous execution of tasks, essential for real-time systems like online voting platforms. Security is a core focus of Java, with features like bytecode verification, sandboxing, and access control mechanisms embedded within the JVM. Java applications also benefit from a rich set of APIs that facilitate everything from network communication to data encryption and graphical user interface development. Java's popularity is further strengthened by its extensive library

support and achive developer community. The Java platform comprises the Java Language, Java API, and JVM. The API includes predefined classes and interfaces organized into packages that developers can use to build applications. Examples include java.io for input/output operations and java.net for networking. Developers can write Java programs.that are easily integrated into client-server architectures or enterprise-level systems. Java applications are compiled into bytecode using the Java compiler. This bytecode is executed by the JVM, making the program portable and efficient. Java applets, servlets, and applications serve different roles in software development. While applets run in browsers, servlets are server-side components used to generate dynamic web content. Overall, Java's robustness, portability, and scalability make it a strategic choice for implementing secure and interactive web applications. It continues to evolve, with frameworks like Spring and Hibernate extending its capabilities.

- **JDBC (Java Database Connectivity):**

JDBC is a Java API that enables interaction between Java applications and relational databases. It provides a standardized method for connecting to and executing queries on databases, making it a crucial component for any data-driven. Java application. JDBC allows developers to perform basic database operations such as creating, reading, updating, and deleting records (CRUD). These operations are executed using SQL statements embedded within the Java code. JDBC provides interfaces and classes for managing database connections, issuing queries, and processing the results. JDBC drivers are essential for facilitating the connection between Java applications and databases. There are four types of JDBC drivers: JDBC-ODBC bridge driver, native-API driver, network protocol driver, and thin driver. Most modern applications use the thin driver due to its portability and performance. The main interfaces in JDBC include Driver Manager, Connection, Statement, Prepared Statement, and ResultSet. Driver Manager is used to establish a connection, Statement and Prepared Statement are used to execute SQL commands, and ResultSet is used to retrieve query results. Prepared Statements also help prevent SQL injection by allowing parameterized queries. JDBC supports transactions, which are essential for maintaining data consistency. Developers can group multiple operations into a single transaction, which either fully completes or is rolled back in case of an error. JDBC also supports batch processing to execute multiple statements efficiently. Exception handling in JDBC is facilitated through SQL Exception which provides detailed information about

database access errors. Logging and debugging JDBC connections help in identifying performance bottlenecks and ensuring smooth operation. JDBC integrates seamlessly with various relational databases like MySQL, Oracle, and PostgreSQL. It also supports metadata retrieval, enabling applications to query the structure of the underlying database schema. In this system, JDBC is used to manage user credentials, voting records, and QR code tokens. It ensures that data operations are reliable, secure, and consistent with the business logic of the application. JDBC's versatility and robustness make indispensable.

- **ODBC (Open Database Connectivity):**

ODBC is a standard API developed by Microsoft that allows applications to access data in a variety of database management systems (DBMS) using SQL. It provides a universal data access interface, making it easier for developers to write database-agnostic code. ODBC achieves platform and DBMS independence by using drivers that translate the application's data queries into commands understood by the target database. This abstraction layer simplifies the development process and enhances compatibility across diverse systems. Through the ODBC Data Source Administrator in Windows, developers can define and configure data sources, specifying the database location, driver, and connection settings. These data sources act as logical links between the application and the physical database. ODBC supports structured and complex SQL queries, transactions, and stored procedures. It can handle both 16-bit and 32-bit data sources, maintaining separate data source names (DSNs) for each. DSNs are identifiers that associate a name with a specific database. One of the key benefits of ODBC is its ability to interact with multiple types of data sources, including relational databases, spreadsheets, and even flat files. It provides a consistent API, so the same application code can be used to access different databases with minimal changes. ODBC drivers are responsible for handling communication between the application and the DBMS. These drivers are loaded dynamically based on the DSN and provide error-handling capabilities to manage exceptions and connectivity issues. In client- server environments, ODBC can manage network communications, reducing the complexity for application developers. It abstracts low-level details, allowing focus on business logic rather than connection mechanics. Although not as efficient as direct native access, ODBC is highly beneficial in environments where flexibility and cross-database compatibility are important. Its use in this system ensures broader adaptability and potential for integration with future platforms. The combination of ODBC with JDBC provides developers with multiple pathways to manage data connections, ensuring both robustness and scalability. It enhances the system's ability

to work across different configurations and environments. ODBC is especially valuable in enterprise settings where data sources may vary and integration with legacy systems is required. Its enduring relevance lies in its simplicity and the widespread availability of drivers.

- **MySQL Database:**

MySQL is a widely used, open-source relational database management system (RDBMS) that supports multi-user access and structured query language (SQL). It is recognized for its performance, reliability, and ease of use, making it a preferred choice for many web- based and enterprise-level applications. MySQL enables developers to organize and manage data efficiently through tables, rows, columns, and relationships. Each table contains rows of data entries and columns that define data attributes. Primary keys and foreign keys are used to enforce data integrity and relational connections between tables. The system supports various SQL operations such as SELECT, INSERT, UPDATE, and DELETE. These operations allow seamless interaction with stored data, ensuring that data retrieval and modification are accurate and efficient. MySQL also provides indexing and full-text search capabilities for optimized performance. Security in MySQL is managed through user accounts, passwords, and privileges. Users can be assigned different access levels to databases and tables, ensuring only authorized users perform specific actions. SSL encryption further secures data transmission between client and server. MySQL can handle large databases efficiently, with support for millions of rows and high concurrency. It is optimized for read-heavy workloads and is compatible with multiple storage engines like InnoDB and MyISAM. InnoDB supports transactions, foreign keys, and ACID compliance, ensuring reliability and data consistency. Developers can interact with MySQL through graphical tools like MySQL Workbench or via command-line interface.

MySQL supports programming languages like Java, PHP, Python, and C/C++, offering flexibility in application development. Integration with tools like JDBC and ODBC enhances its usability across platforms. Backup and recovery in MySQL can be managed using utilities such as MySQL dump and mysqlhotcopy. The system also supports replication for high availability and fault tolerance, with master-slave and master-master configurations available. In this project, MySQL serves as the backbone for storing sensitive voting data, including voter details, authentication tokens, election configurations, and voting logs. Its stability and scalability ensure reliable performance under variable workloads. MySQL continues to be actively developed and supported by Oracle, with a large community contributing to its growth.

- **JFreeChat**

JFreeChart is a comprehensive, open-source Java library that facilitates the creation of professional-quality charts and graphical representations of data. It supports a variety of chart types, making it a suitable tool for presenting election results and analytical reports within the voting system. The library supports different chart formats including bar charts, pie charts, line charts, time series, XY plots, and more. These charts can be customized with labels, colors, legends, and tooltips to enhance readability and user interaction. JFreeChart is widely used in both desktop and web-based applications. JFreeChart is designed with a modular and extensible architecture. Developers can easily embed charts into Java Swing components, servlet-based applications, or export them as image files in formats like PNG and JPEG. It also supports output in vector formats like PDF and SVG for high-resolution printing. Integration with Java applications is seamless, thanks to its clean API and comprehensive documentation. JFreeChart handles dynamic data updates, making it ideal for applications requiring real-time visualization of changing datasets, such as live election result dashboards. The library supports interactive features such as zooming, panning, and data point selection. It also allows annotation and event handling, enabling users to extract deeper insights from visual data presentations. For this project, JFreeChart is employed to graphically represent voting statistics, such as vote counts per candidate, voting trends over time, and turnout analysis. These visualizations aid both administrators and users in understanding complex data quickly. The library includes built- in support for data sources such as datasets and collections, and developers can create custom datasets to suit application-specific requirements. It is compatible with various Java development environments and platforms.JFreeChart's LGPL licensing allows its use in both open-source and proprietary applications, making it a flexible choice for diverse software projects. Its efficiency and flexibility ensure that users receive timely, meaningful insights into system data. As the project scales or adds more analytical features, JFreeChart's capabilities can be extended through plug-ins or custom renderers. It plays a critical role in exsuring the system is informative, user-friendly, and transparent in displaying results.

5.2 Code:

add_candidate.java

```
package QRVoting
 import java.io.IOException;
 import java.io.InputStream;
 import java.io.PrintWriter;
 import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
 import java.sql.Statement;
import javax.servlet.ServletException;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.Part; @MultipartConfig(maxFileSize = 16177215)
public class add_candidate extends HttpServlet {
protected void processRequest(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
response.setContentType("text/html;charset=UTF-8");
try (PrintWriter out = response.getWriter()) {
 /* TODO output your page here. You may use following sample code. */
  String election_name = request.getParameter("election_name");
  String candidate_name = request.getParameter("candidate_name");
  String party_name = request.getParameter("party_name");
  String address = request.getParameter("address");
   String phone = request.getParameter("phone");
   InputStream fis = null;
   Part filePart = request.getPart("photo");
   if (filePart != null) {
  // prints out some information for debugging
   System.out.println(filePart.getName());
```

```
System.out.println(filePart.getSize());
System.out.println(filePart.getContentType());// obtains input stream of the upload file
  fis = filePart.getInputStream();
   }
System.out.println("Election Name :" + election_name + "Candidate name :" +
candidate_name + "Address :" + address + "Phone No :" + phone);
Connection conn = SQLconnection.getconnection();
   String message = null;
    try {
    Statement st = conn.createStatement();
    String sql = "insert into add_candidate(election_name, candidate_name, address, phone,
photo, party_name) values (?, ?, ?, ?, ?, ?)";
    PreparedStatement statement = conn.prepareStatement(sql);
    statement.setString(1, election_name);
     statement.setString(2, candidate_name);
      statement.setString(3, address);
       statement.setString(4, phone);
         if (fis != null)
statement.setBlob(5, fis);
            }
            statement.setString(6, party_name);
            int row = statement.executeUpdate();
            if (row > 0) {
                response.sendRedirect("Add_candidate.jsp?Candidate_added");
            } else {
                response.sendRedirect("Add_candidate.jsp?Failed");
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
           }
        }

// <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign
```

```
on the left to edit the code.">
    /**
     * Handles the HTTP <code>GET</code> method.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    /**
     * Handles the HTTP <code>POST</code> method.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
    /**
     * Returns a short description of the servlet.
     *
     * @return a String containing servlet description
      */
    @Override
```

```
    public String getServletInfo() {
        return "Short description";
    }// </editor-fold>


}
```

add_election.java

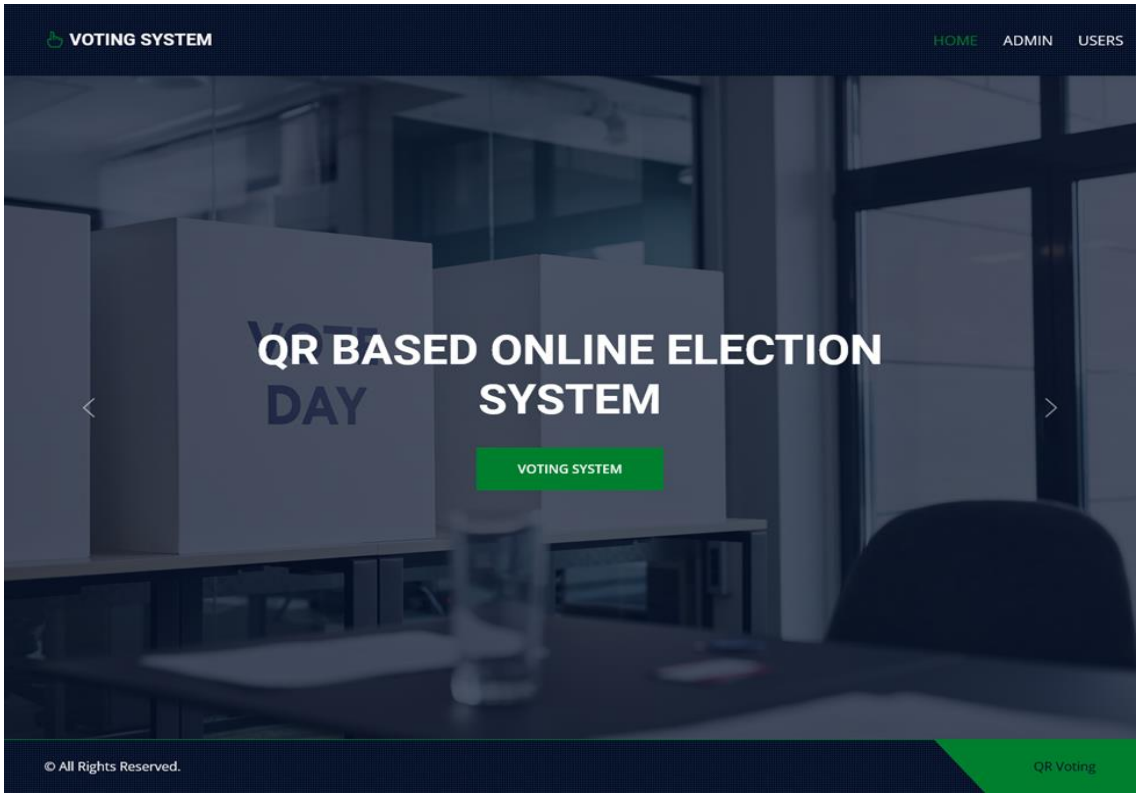
```
package QRVoting;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
public class add_election extends HttpServlet {
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
response.setContentType("text/html;charset=UTF-8");
try (PrintWriter out = response.getWriter()) {
String election_name = request.getParameter("election_name");
String end_date = request.getParameter("end_date");
System.out.println("Election Name :" + election_name +  "End Date"+ end_date);
 Connection conn = SQLconnection.getconnection();
  String message = null;
   try {
   Statement st = conn.createStatement();
   ResultSet rs = st.executeQuery("Select * from add_election where election_name ='" +
   election_name + "'");
    if (rs.next()) {
    response.sendRedirect("User_login.jsp?msg=Election_name_Exists");
```

```
} else {
    String sql = "insert into add_election(election_name, end_date) values (?,?)";
    PreparedStatement statement = conn.prepareStatement(sql);
    statement.setString(1, election_name);
    statement.setString(2, end_date);
    int row = statement.executeUpdate();
    if (row > 0) {
        response.sendRedirect("Add_election.jsp?Election_added");
    } else {
        response.sendRedirect("Add_election.jsp?Failed");
    }
                }
            } catch (SQLException ex) {
                ex.printStackTrace();
            }


        }
    }
    // <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on
the left to edit the code.">
    /**
       * Handles the HTTP <code>GET</code> method.
       *
       * @param request servlet request
       * @param response servlet response
       * @throws ServletException if a servlet-specific error occurs
       * @throws IOException if an I/O error occurs
       */
      @Override
      protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
         processRequest(request, response);
  }
```

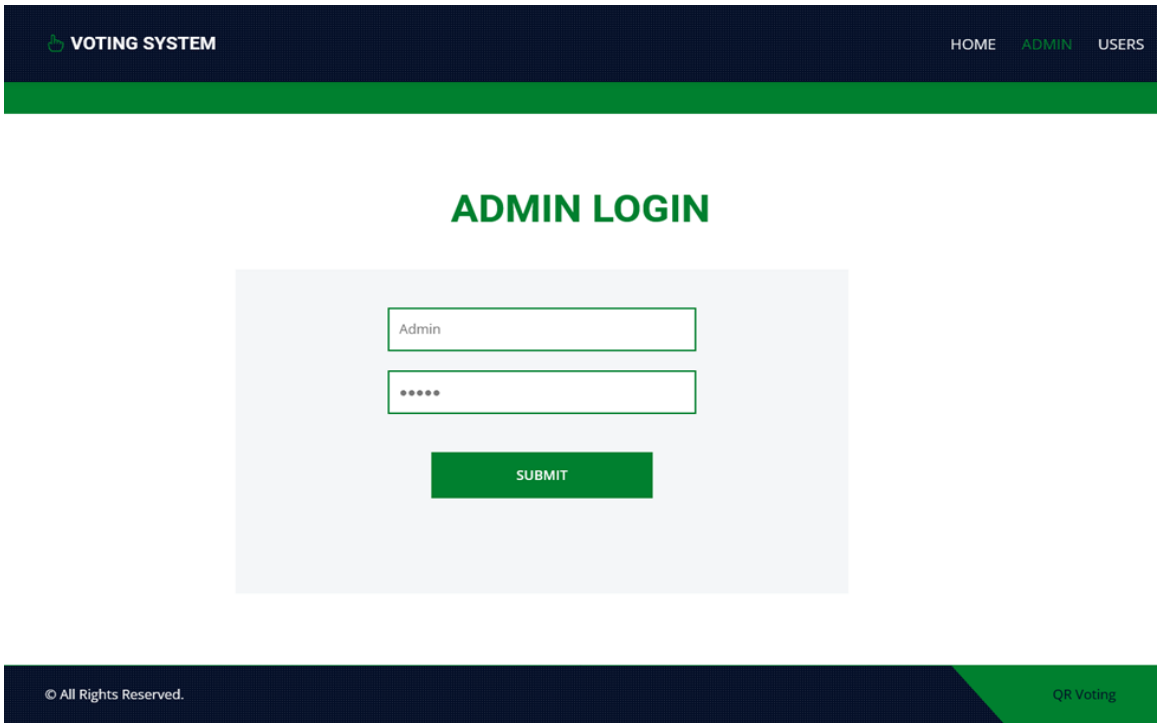
```
    /**
     * Handles the HTTP <code>POST</code> method.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    /**
     * Returns a short description of the servlet.
     *
     * @return a String containing servlet description
     */
    @Override
    public String getServletInfo() {
        return "Short description";
    }// </editor-fold>

}
```

6. Screenshots

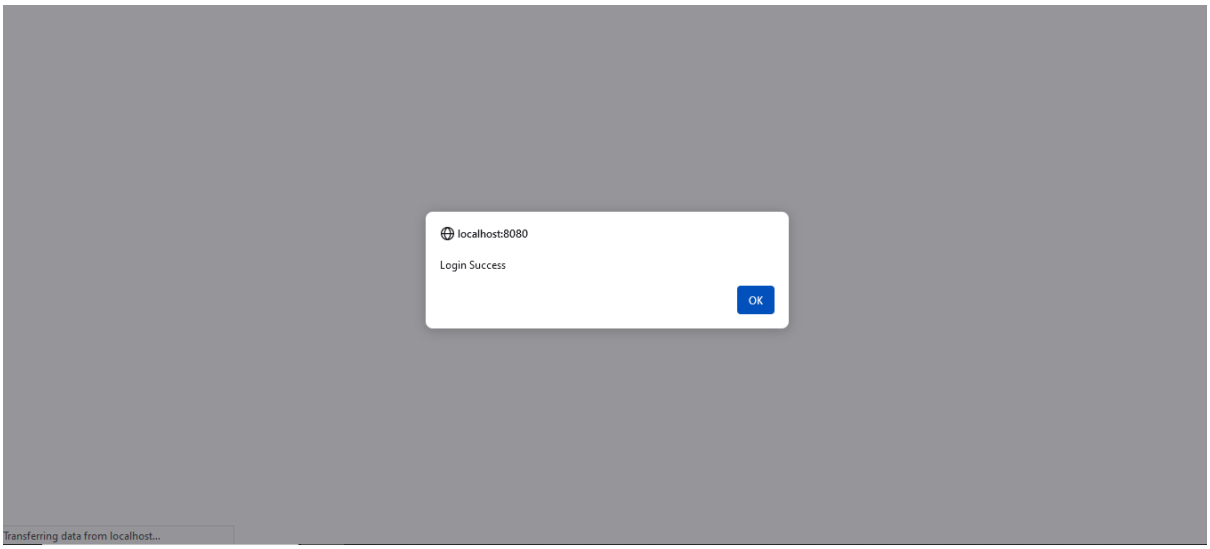
6.Screenshots



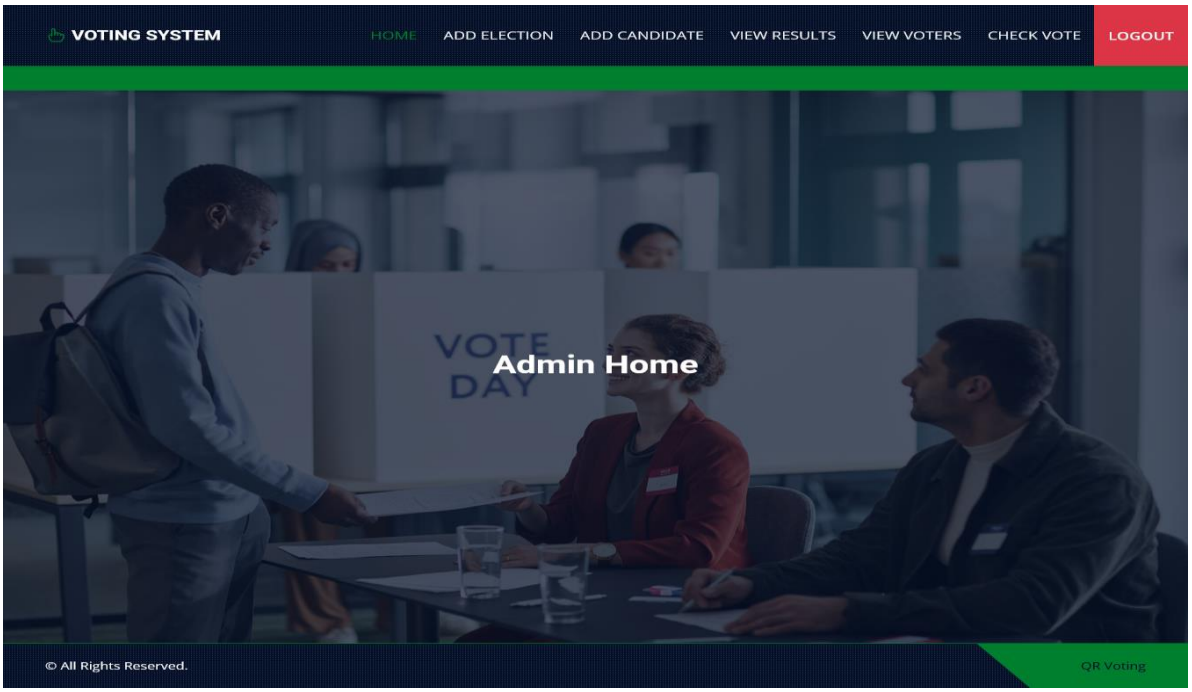
Screenshot 6.1: Home Page



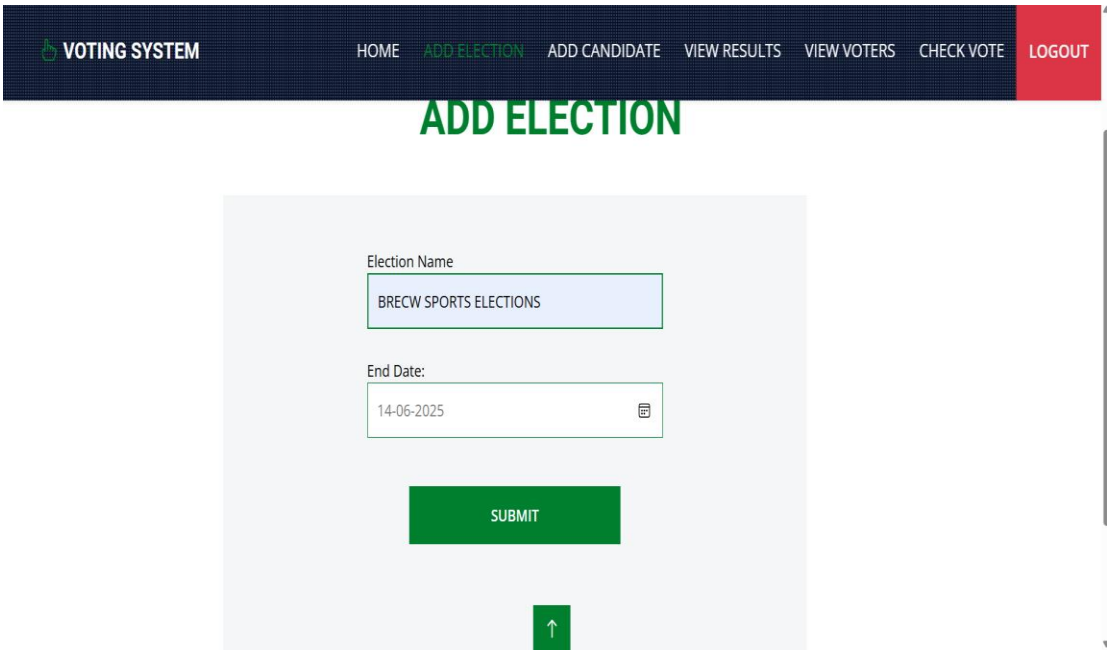
Screenshot 6.2: Admin Login



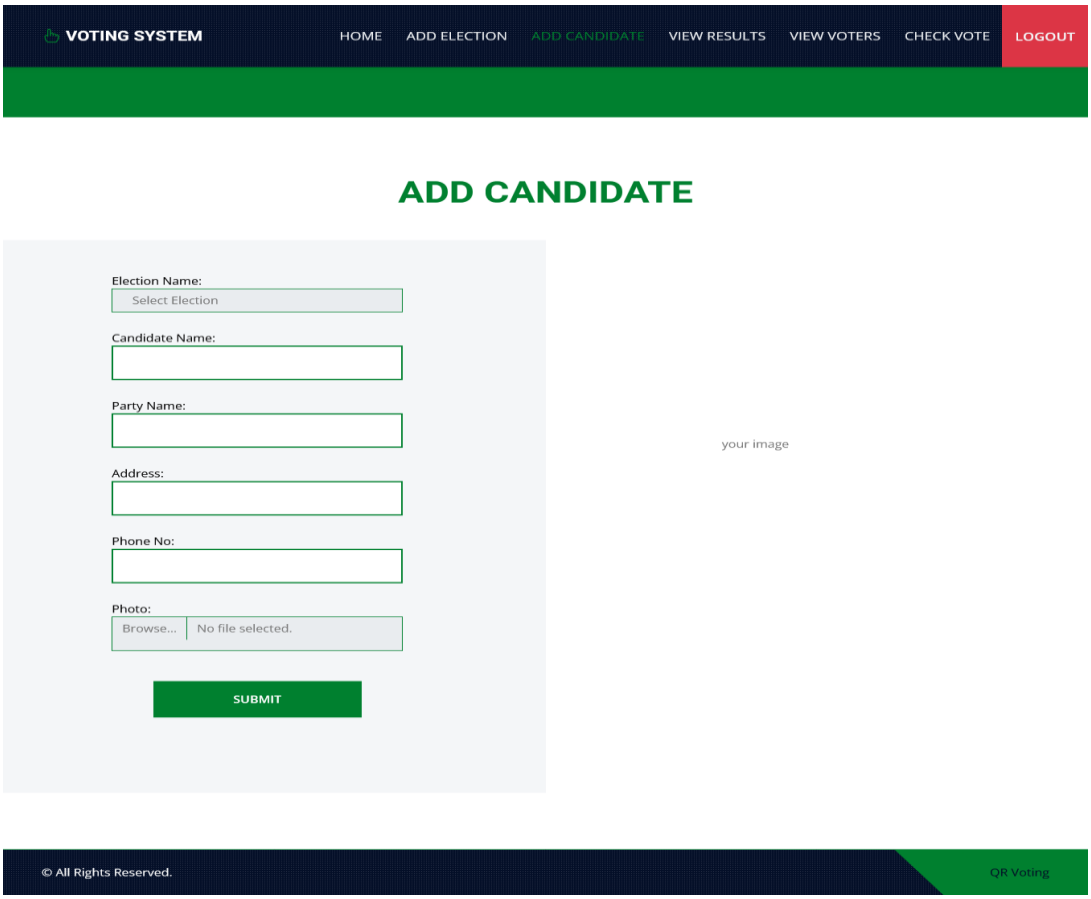
Screenshot 6.3: Login Success



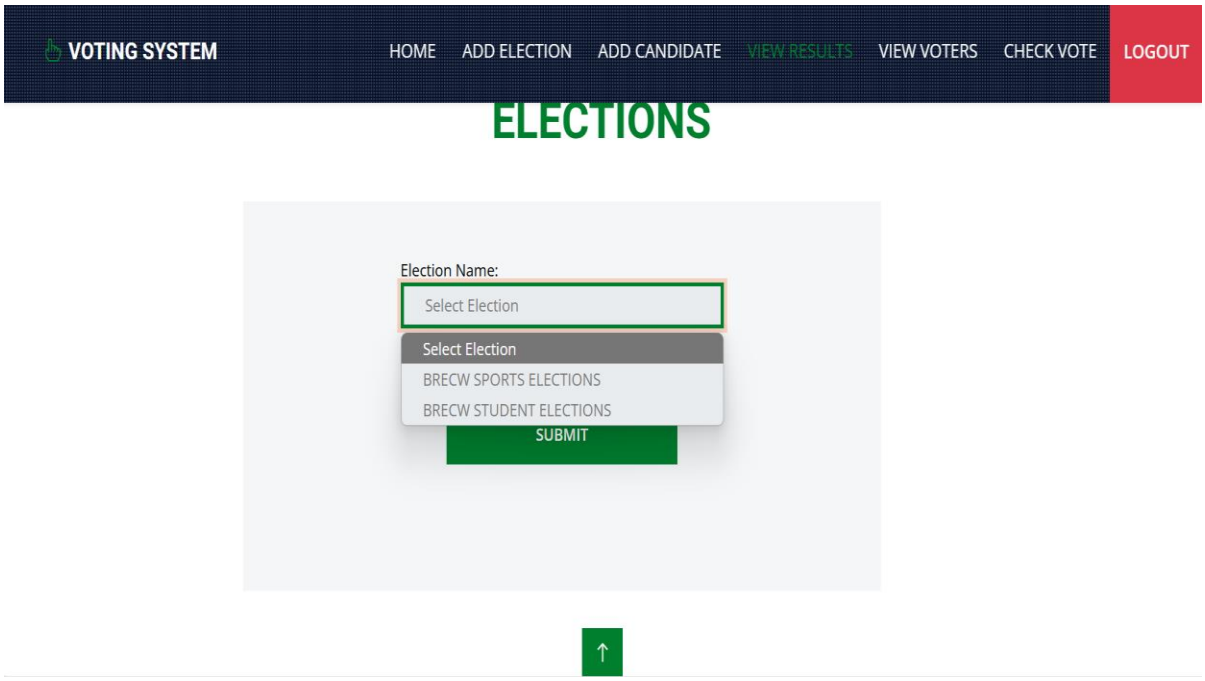
Screenshot 6.4: Admin Home



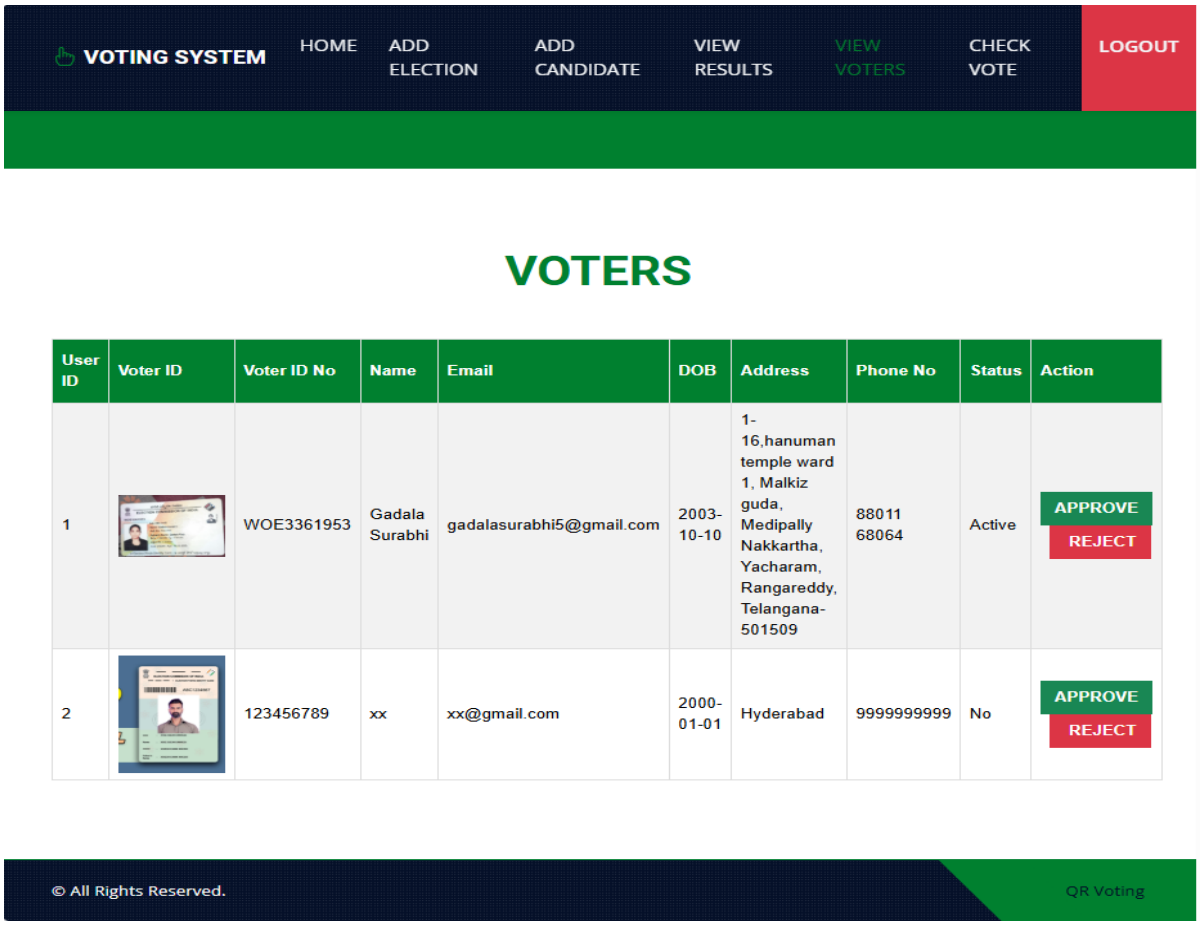
Screenshot 6.5: Add Election



Screenshot 6.6: Add Candidate

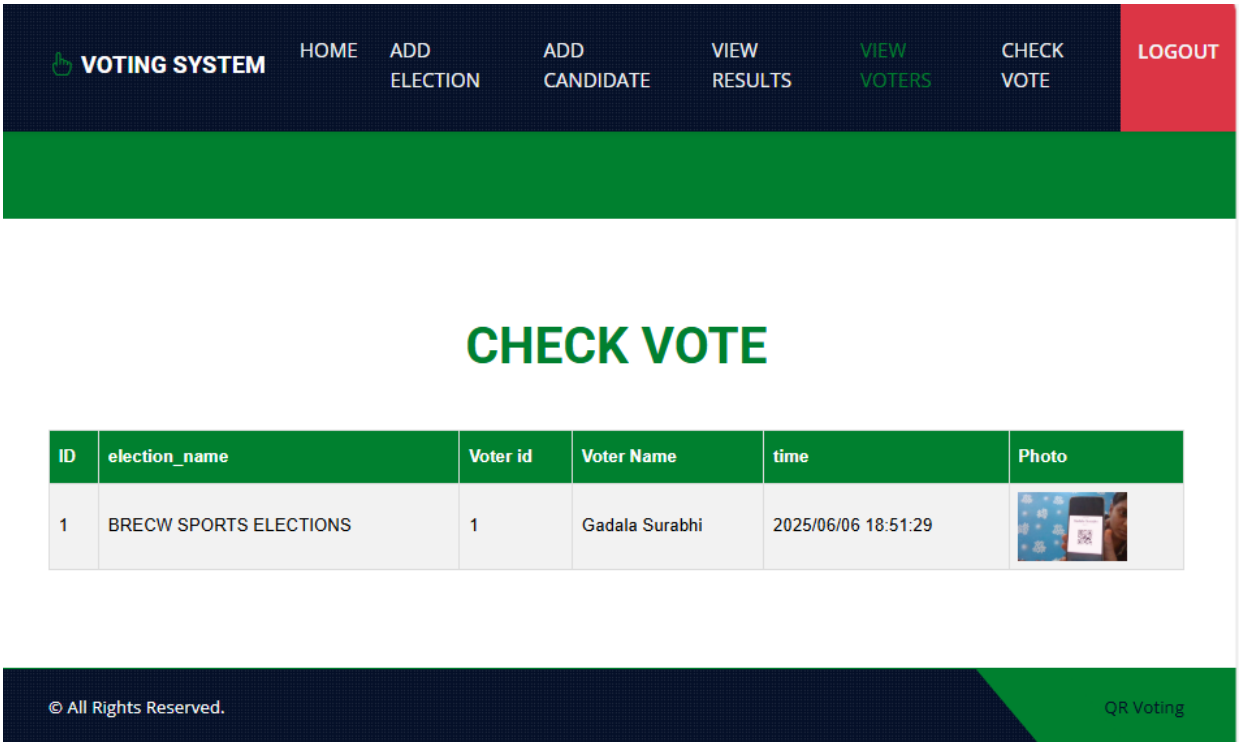


Screenshot 6.7: Election View Result

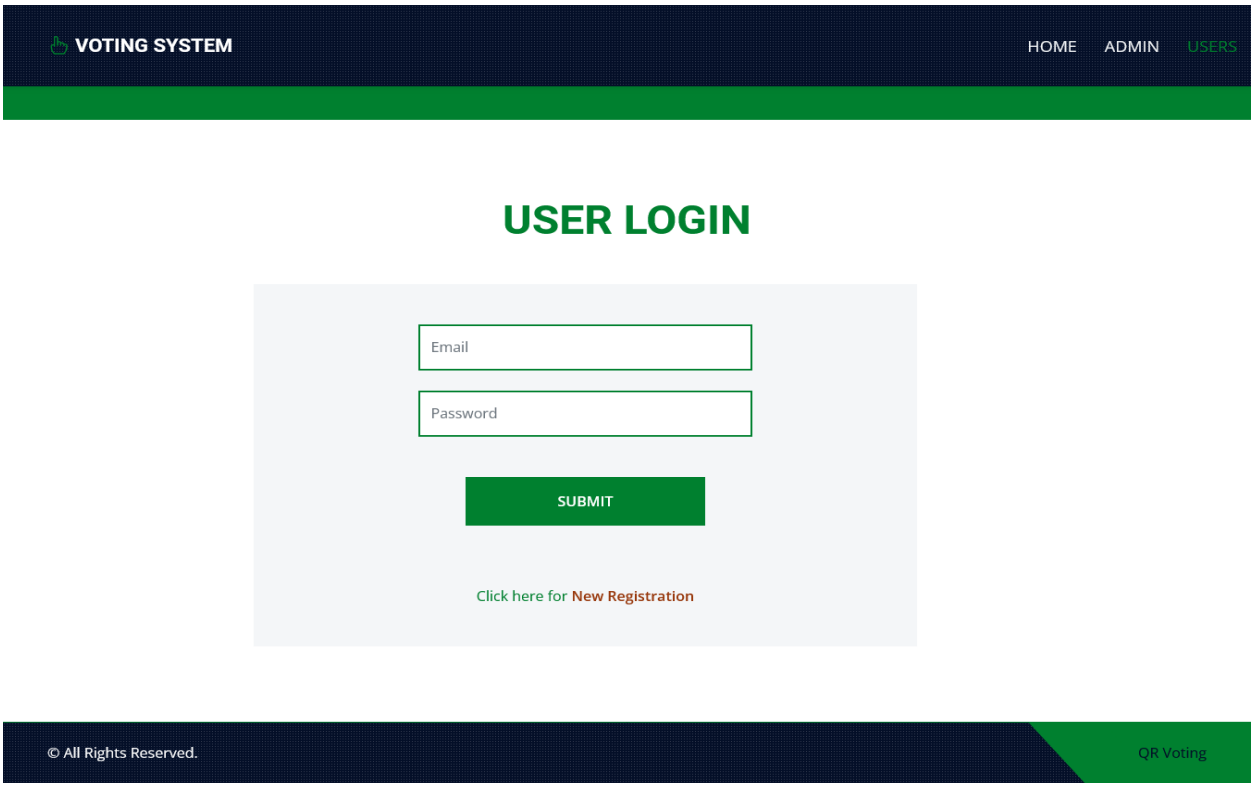


User ID	Voter ID	Voter ID No	Name	Email	DOB	Address	Phone No	Status	Action
1		WOE3361953	Gadala Surabhi	gadalasurabhi5@gmail.com	2003-10-10	1-16,hanuman temple ward 1, Malkiz guda, Medipally Nakkartha, Yacharam, Rangareddy, Telangana-501509	8801168064	Active	APPROVE REJECT
2		123456789	xx	xx@gmail.com	2000-01-01	Hyderabad	9999999999	No	APPROVE REJECT

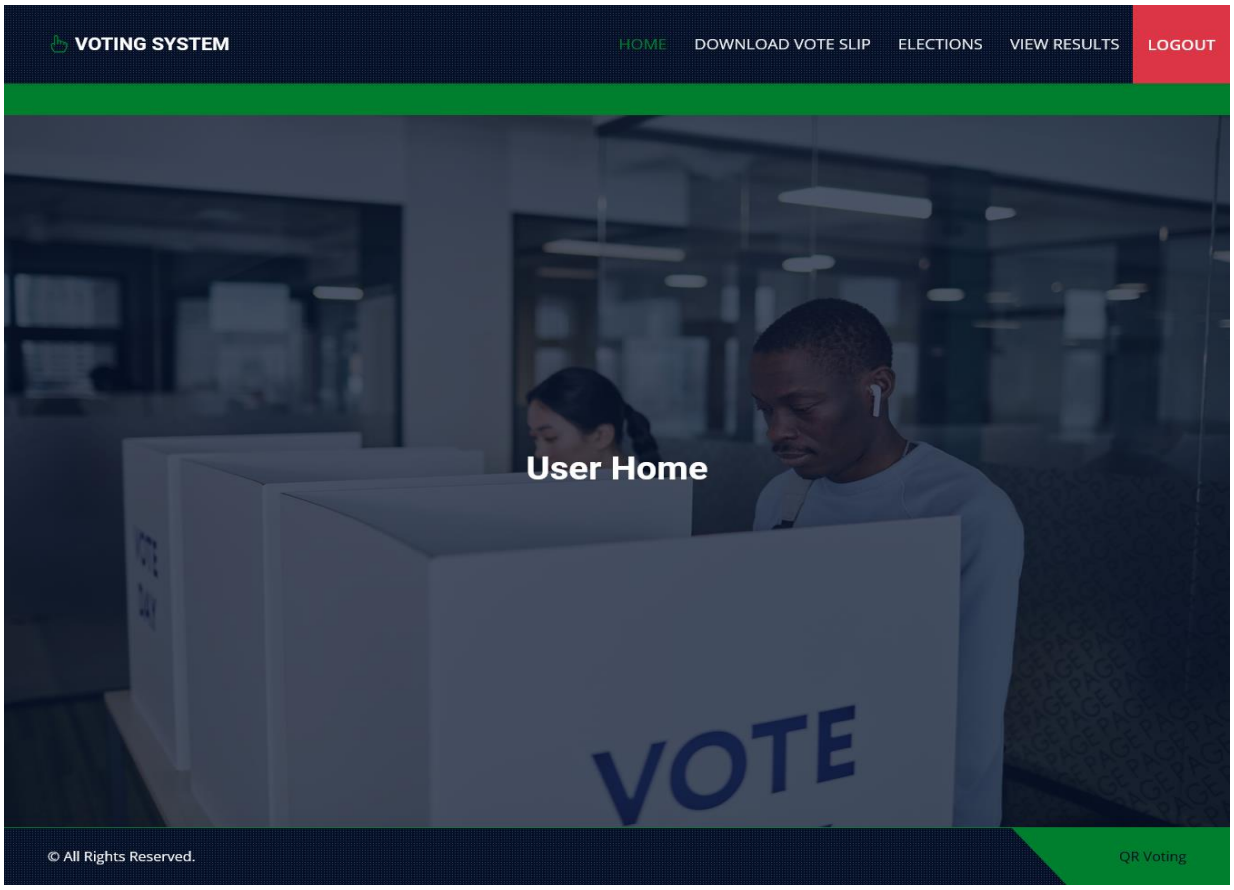
Screenshot 6.8: View Voters



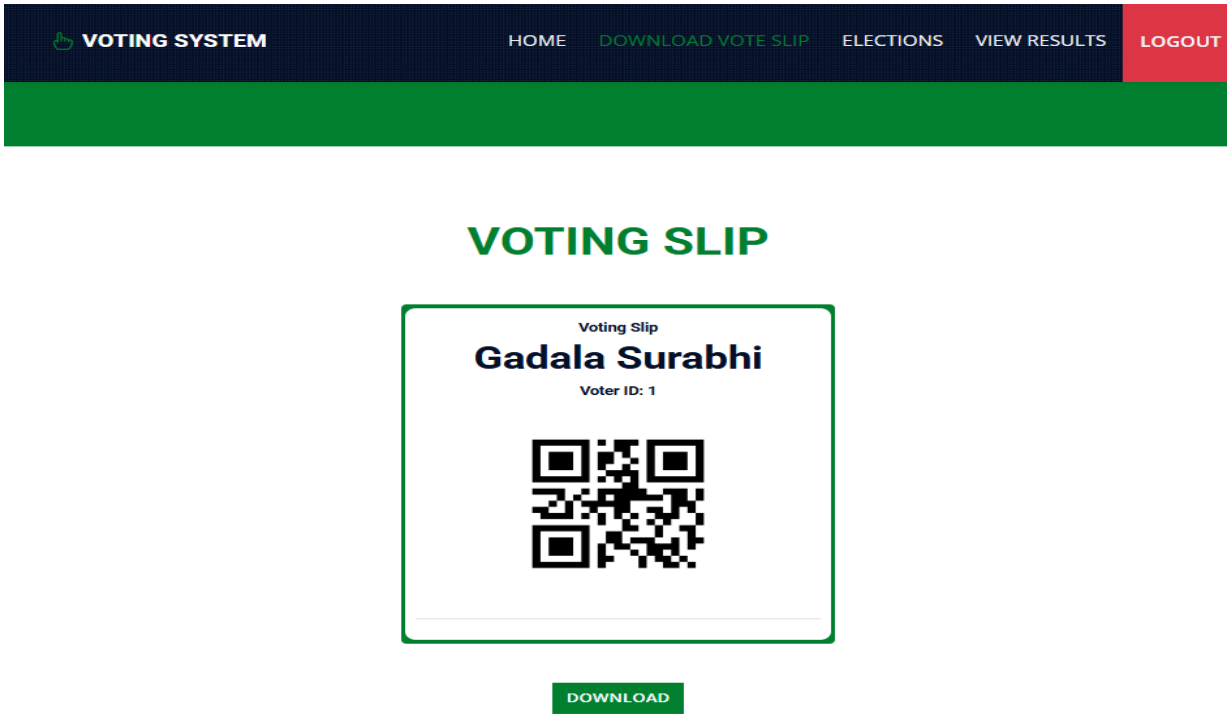
Screenshot 6.9: Check Vote



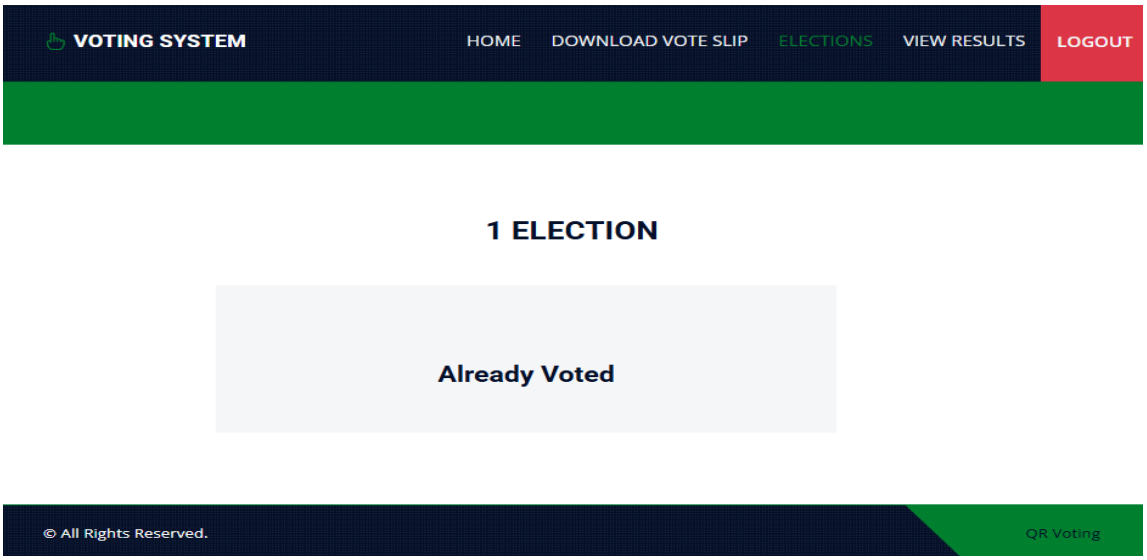
Screenshot 6.10: User Login



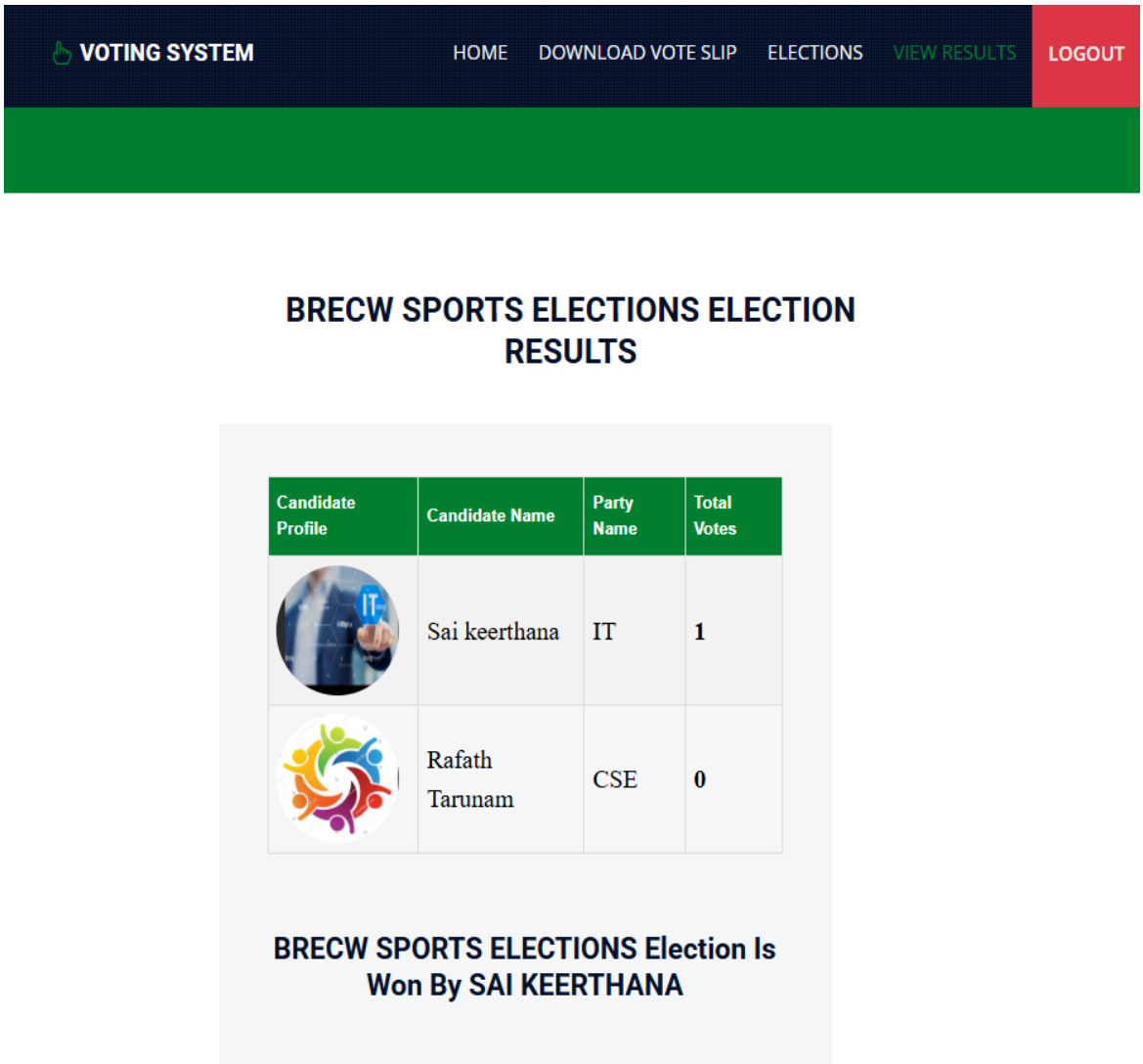
Screenshot 6.11: User Home



Screenshot 6.12: Voting slip



Screenshot 6.13: Election voting page through webcam



Screenshot 6.14: View Results page

7. Testing

7. Testing and Validations

7.1 Test Cases:

7.1.1 Test Strategy:

Field testing will be performed manually and functional tests will be written in detail.

Features to be tested

- Verify that we get proper output
- Verify that all painters and eraser work

The actual purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner.

7.2 Types of Testing:

System testing is a critical phase in the software development life cycle that focuses on assessing the overall quality, functionality, and performance of a software system. It is a comprehensive and systematic process that aims to identify defects, ensure that the system meets specified requirements, and verify its readiness for deployment. System testing plays a crucial role in delivering reliable, robust, and high- quality software solutions.

7.2.1 Unit Testing:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results. Unit testing is an essential practice in software development that involves testing individual units or components of a software application in isolation. Each unit, typically a small piece of

code or a function, is tested to ensure that it functions correctly and produces expected outcomes. Unit testing plays a pivotal role in maintaining code quality, catching bugs early, and facilitating efficient debugging and maintenance.

7.2.2 Integration Testing:

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

7.2.3 Functional Testing:

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked. Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

7.2.4 System Testing:

System testing ensures that the entire integrated software system functions as intended and meets both functional and non-functional requirements. It is conducted in a complete, end- to-end environment that closely simulates production conditions. The purpose of system

testing is to validate the system as a whole and verify that it behaves correctly when all individual components and modules are combined and operating together.System testing evaluates the application's workflow, business logic, external interfaces, data integrity, and user interactions. It typically involves executing the application with a variety of input scenarios, including expected, boundary, and invalid data, to ensure reliable and consistent performance. This phase is crucial because it identifies issues that may not be apparent during unit or integration testing.

7.2.5 White Box Testing:

White Box Testing, also known as clear box testing, glass box testing, or structural testing, is a software testing technique in which the tester has complete knowledge of the internal structure, logic, and source code of the application. Unlike black box testing, which focuses on testing the software from an external or user-facing perspective, white box testing allows the tester to “look inside” the software and validate the internal operations.

This type of testing is particularly useful for verifying control flow, data flow, decision branches, loops, and logical paths within the code. Testers often write test cases based on the source code to ensure that all possible paths are executed and work correctly. White box testing can uncover hidden bugs such as logic errors, incorrect assumptions, and security vulnerabilities that may not be detected through external testing methods.

7.2.6 Black Box Testing:

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box you cannot “see” into it.

The test provides inputs and responds to outputs without considering how the software works. Black box testing is a testing methodology that focuses on evaluating the functionality of a software application without examining its internal code, structure, or implementation details. Instead, this approach treats the software as a "black box," where the tester interacts with the application's inputs and examines its outputs to assess whether the desired functionalities work as expected. Black box testing emphasizes validating the software's behavior based on user specifications, requirements, and expected outcomes.

Test Cases

S. No	Test Case Scenario	Expected Output	Actual Output	Result
1	QR Code Generation for Voter ID	QR code generated for registered voter	QR code generated successfully	Pass
2	QR Code Scanning	Voter details displayed after successful scan	Voter details displayed correctly	Pass
3	Invalid QR Code	Error message: "Invalid QR Code"	Error message not shown, system crashed	Fail
4	Login with Valid voter credentials	Redirect to voting dashboard	Dashboard accessed	Pass
5	Login with invalid credentials	Display "Invalid credentials" message	Login error shown	Pass
6	Vote casting (valid, not voted before)	Confirmation message: "Vote cast successfully"	Vote recorded and confirmation shown	Pass
7	Duplicate voting attempt	Error message: "Vote already cast"	System allowed voting again	Fail
8	Admin login and access	Admin redirected to admin panel	Adminpanel accessed	Pass
9	View vote count as admin	Vote counts displayed per candidate	Correct vote counts shown	Pass
10	Logout functionality	User redirected to login page	Successfully redirected	Pass

11	Session timeout after inactivity	Auto-logout and return to login screen	Session did not expire after timeout	Fail
12	Network failure during voting	Alert message and retry option	Retry message displayed	Pass
13	Mobile responsiveness check	App adjusts layout to fit mobile screen	Mobile display functional	Pass
14	Unauthorized admin panel access attempt	Access denied, redirected to login	Unauthorized access blocked	Pass
15	View voting results after end of election	Final results displayed to admin	Accurate final results shown	Pass

Table 1: Test Cases

7.3 Result Analysis:

The result analysis of the QR Code-Based Online Voting System shows that it effectively manages key voting processes, including QR code generation, user authentication, vote casting, result display, and administrative operations. The system demonstrated overall stability, successfully passing 12 out of 15 test cases, with reliable performance under standard conditions.

In addition to its core functionality, the system highlighted areas requiring improvement to enhance security and user experience. Specifically, it failed to handle invalid QR codes gracefully, leading to crashes instead of appropriate error messages. A critical vulnerability was identified in vote integrity, as the system permitted duplicate votes from the same voter, compromising election fairness. Additionally, the session timeout mechanism did not enforce proper user logouts, allowing extended access beyond the defined inactivity threshold. Addressing these issues is essential to ensure robustness and trustworthiness in digital voting environments.

From a technical standpoint, the platform a solid foundation but requires targeted debugging and refinement. Enhancements in error handling, duplicate vote prevention, and session management will strengthen security and system integrity. With necessary improvements, this QR Code-Based Online Voting System holds great potential as a reliable and secure digital voting platform.

8. Conclusion

8. Conclusion

The QR Code-Based Secure Online Voting System successfully achieves its goal of providing a secure, accessible, and efficient voting solution by leveraging QR code authentication and a robust backend architecture built with Java, JSP, HTML, CSS, JavaScript, and MySQL. It streamlines the voting process through integrated modules for user registration, QR code management, vote casting, and result reporting. The system ensures voter accessibility through remote participation while maintaining strong security measures to prevent fraud and unauthorized access. By enforcing single-vote integrity per election, it upholds electoral fairness and trust in the voting system. The platform's support for transparent result reporting, real-time voter authentication, and administrative oversight further enhances its reliability in digital elections. Its strong performance in usability, security, and efficiency makes it suitable for official electoral processes, remote voting applications, and institutional voting systems. With future enhancements, the system has the potential to evolve into a fully scalable and secure digital voting platform, incorporating advanced features such as blockchain integration, biometric authentication, a mobile app, and multilingual support. Additional improvements in scalability, voter education tools, government database integration, and diverse voting method support will further refine its functioality and inclusivity, ensuring broader adoption and enhanced election integrity.

9. Future Scope

9. Future Scope

In the future, a QR Code-Based Online Voting System could be scaled up to handle large national and state elections, offering real-time performance even with millions of voters. By integrating biometric authentication methods like fingerprint or facial recognition, the system can make voter identity checks much more secure. To boost transparency and trust, blockchain technology could be used to store votes in a tamper-proof way. The system should also support multiple languages and include accessibility features for differently-abled users, ensuring everyone can participate easily. AI-powered analytics could help spot unusual voting patterns or possible fraud, making the process safer. A mobile app would make voting more convenient and accessible, even from remote locations. For areas with poor internet, offline voting using encrypted QR codes could allow everyone to vote securely. Deploying the system on the cloud would improve its reliability and make disaster recovery easier. Working closely with government election bodies could pave the way for official adoption. Finally, regular security audits and penetration testing would help keep the system safe and trustworthy in the long run.

10. References

10. References

[1] A. Sankara Naarayanan, "QR Codes and Security Solutions," International Journal of Computer Science and Telecommunications, [volume 3, Issue 7, July 2012].

[2] Prashanth P. Bungale and Swaroop Sridhar, "Electronic Voting Survey."

[3] Muhammad Nadeem, Dr. Javaid, R. Laghari, SZABIST Study of Pakistan election System as "Intelligent e-Election," SZABIST/PhD.

[4] Roger Koenker and Kevin F. Hallock, "Quantile Regression", Journal of Economic Perspectives-Volume 15, Number 4-Fall 2001-Pages 143-156.

[5] Peter Kieseberg, Manuel Leithner, Martin Mulazzani, Lindsay munroe, Sebastian Schrittwieser, Mayank Sinha, Edgar Weippl, "QR Code Security," Favoritenstrasse 16, AT-1040 vienna, Austria.

[6] Shobha lokhande, Dipali sawant, Nazneen Sayyad, Mamata Yengul, D.D. Pukale, "E-Voting through Biometrics and Cryptography Steganography Technique with Conjuction of GSM Modem," Emerging Trends in computer Science and InformationTechnology2012(ETCSIT2012), Proceedings Published in International Journal of Computer Applications(IJCA).

[7] Orhan Cetinkaya and Mehmet Levent Koc, "Practical Aspects of DynaVote e-Voting Protocol," Electronic journal of Government Volume 7 Issue 4 2009(pp327-338).

[8] Sanjay Kumar, Manpreet Singh, "Design a Secure Electronic Voting System Using Fingerprint Technique," IJCSI International Journal of Computer Science Issues, Vol. 10,Issue 4, No 1, July 2013.

[9] Mohamad Abdolahi, Majid Mohamadi, Mehdi Jafari "Multimodal Biometric system Fusion Using Fingerprint and Iris with Fuzzy Logic," International Journel of Soft Computing and Engineering(IJSCE), ISSN: 2231-2307, Volume-2, Issue-6, January 2013.

[10] Malwade Nikita, Patil Chetan, Chavan Suruchi, Prof. Raut S.Y, "Secure Online Voting System Proposed By Biometric And Steganography" , International Journal of Emerging Technology and Advanced Engineering, Website:www.ijetae.com(ISSN 2250-2459, ISO 9001:2008 Certified journal, Volume 3, Issue 5, May 2013).